



Splunk AppDynamics Technical Proposal for **SPF**

Document version 1.0

	1
Splunk AppDynamics Technical Proposal for SPF	1
1.0 Executive Summary and Supplier Name	3
2.0 Platform Overview, Architecture and Features	4
2.1 Splunk AppDynamics Overview	4
2.2 Splunk AppDynamics Architecture	5
2.3 Splunk AppDynamics Features	8
2.3.1 Flow Map	8
2.3.2 Homepage	9
2.3.3 Application Monitoring	11
2.3.4 Business Transaction	14
2.3.5 Deep Infrastructure Monitoring	18
Database Monitoring	21
Network Monitoring	23
Analytics	23
Log Analytics	26
Error Detection	26
Health Rule	27
User Management	28
Dynamics Baseline	29
Service Impact Solution	31
Dashboard & Report	32
Notification	36
Retention Period	38
AppDynamics APIs	39
Integration	40
Security	40
Supported Technologies	41
RoadMap AppDynamics	42
Business Value Analysis	42

1.0 Executive Summary and Supplier Name

The need for innovation and faster software development is driving profound changes in how applications are built and operated. The adoption of microservices architectures, elastic cloud infrastructure (containers, Kubernetes, functions, etc.) and agile DevOps models increases velocity, but also complexity as systems become more dynamic, unpredictable and noisy. Traditional, disjointed monitoring tools can't provide the speed, scale and analytics capabilities needed to support modern digital resiliency like real-time visibility, smart altering and rapid troubleshooting.

Splunk helps organizations (DevOps engineers, SREs and more):

1. **Deliver products faster**

Accelerate the software delivery process for your business objectives more quickly

2. **Improve end user experience**

Catch and fix problems faster to reduce downtime, avoid war rooms and improve performance, end user experience and satisfaction.

3. **Improve operational efficiency and TCO**

Simplify the observability toolchain, get better visibility and control into cloud utilization and right-size your environments

4. **Reduce unplanned work**

Proactively improve code releases and system architecture with system visibility and better tools for monitoring, troubleshooting and incident response

Hereby with this proposal, **Splunk Inc** (orders provided through **DXC Technology Singapore**) would like to thank you for inviting us to take part in this tender and this proposal will outline Splunk Solution offering.

2.0 Platform Overview, Architecture and Features

2.1 Splunk AppDynamics Overview

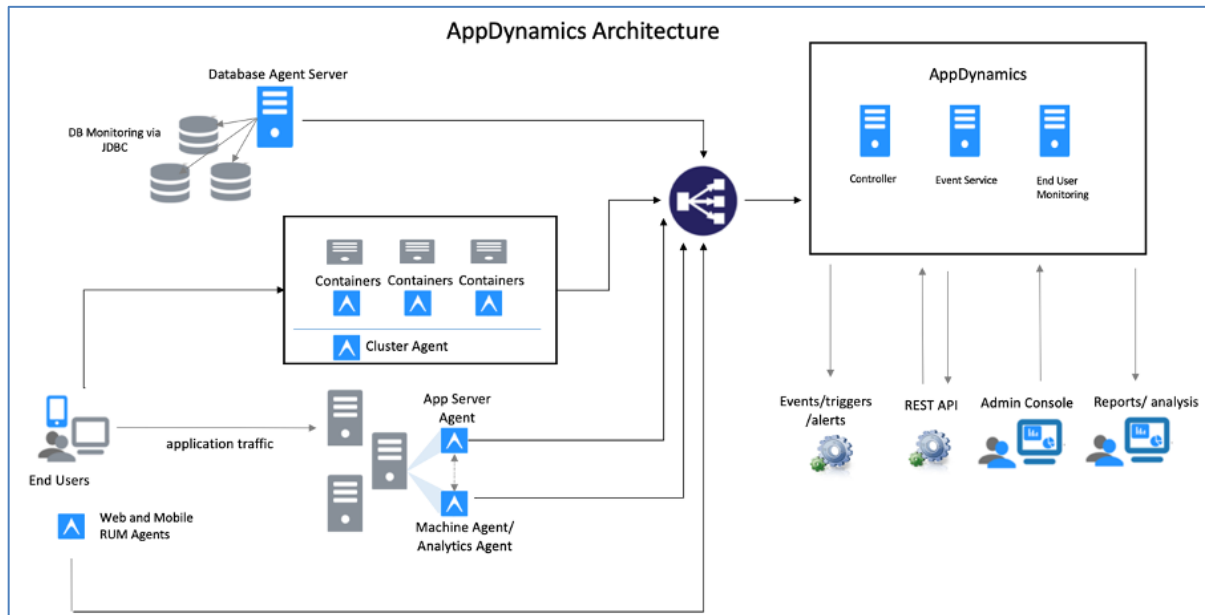
Splunk AppDynamics is a software-based monitoring tools solution that can monitor everything from user experience, infrastructure, network, applications, databases down to the code level in real-time. AppDynamics as a Cisco Full Stack Observability (FSO) Platform is able to maintain the availability and performance of applications and also understand the behavior of each application being monitored.

Not only that, AppDynamics is also able to correlate every part of the stack from the business side, network, application, to infrastructure in single dashboard platform in real time. AppDynamics itself can be implemented in both SaaS and On-Premise environments. AppDynamics also can monitor on-premise application, hybrid cloud application, multiple cloud, and cloud application. for On-Premise features you can upgrade the version periodically to get the latest features.



2.2 Splunk AppDynamics Architecture

AppDynamics has 3 major components to function, namely Controller, Event Service and End User Monitoring Server. These three components have their own functions as referenced at the below table. In the controller component, main page access for web-based AppDynamics is available. This access can be used as the main access for data management, application management agent management, configuration, and all features in AppDynamics.



AppDynamics works by installing lightweight agents on your application servers, database and micro services containers. The agents work for all major application languages such as Java, .NET, PHP, Node.js. The agents will automatically tag, trace and follow application requests to identify important business transactions automatically from the application environment, performance metrics are captured for those business transactions. A dynamic application flow map will also be generated from this process and updated in real time.

Business transactions relate what matters to the end user to application system components. For example, a typical “user login and logout” so called user actions can be identified as business transactions. All business transactions response time are baselined automatically using machine learning. AppDynamics monitors all business transactions on metrics such as response times,

error rate and call per minute. However, only detailed diagnostic snapshots of abnormal transactions are captured for root cause analysis. This helps to keep the overhead low in a production environment.

AppDynamics also provides infrastructure agents to gather metrics such as CPU/Memory/OS process and I/O performance from the system. This information is correlated with application performance for overall monitoring and diagnostics.

AppDynamics consists of a number of components and each component has particular functionality as described in the table below.

Components of AppDynamics:

Components	Role
AppDynamics Controller	AppDynamics Controller is the core of the AppDynamics platform. It receives data from all different agents and performs application performance analytics tasks. Controllers also provided WEB UI for user to access.
AppDynamics Enterprise Console	AppDynamics Enterprise Console is used to deploy and manage the Controller cluster and Event Service Cluster.
AppDynamics Event Service	AppDynamics Event Service Server is a data store which stores analytics event data. In production, three Event Service servers are required to form a high availability cluster.
AppDynamics DB Agent	AppDynamics database agent shall be installed on a dedicated server. Database agents connect to target database (e.g. Oracle DB) through JDBC connection and collect the database performance metrics. The collected data shall be sent to the Controller and Event Service.
AppDynamics Agent	AppDynamics agent collects application performance metrics, code level error, code stack trace from target device and sends it back to AppDynamics Controller and Event Service servers.

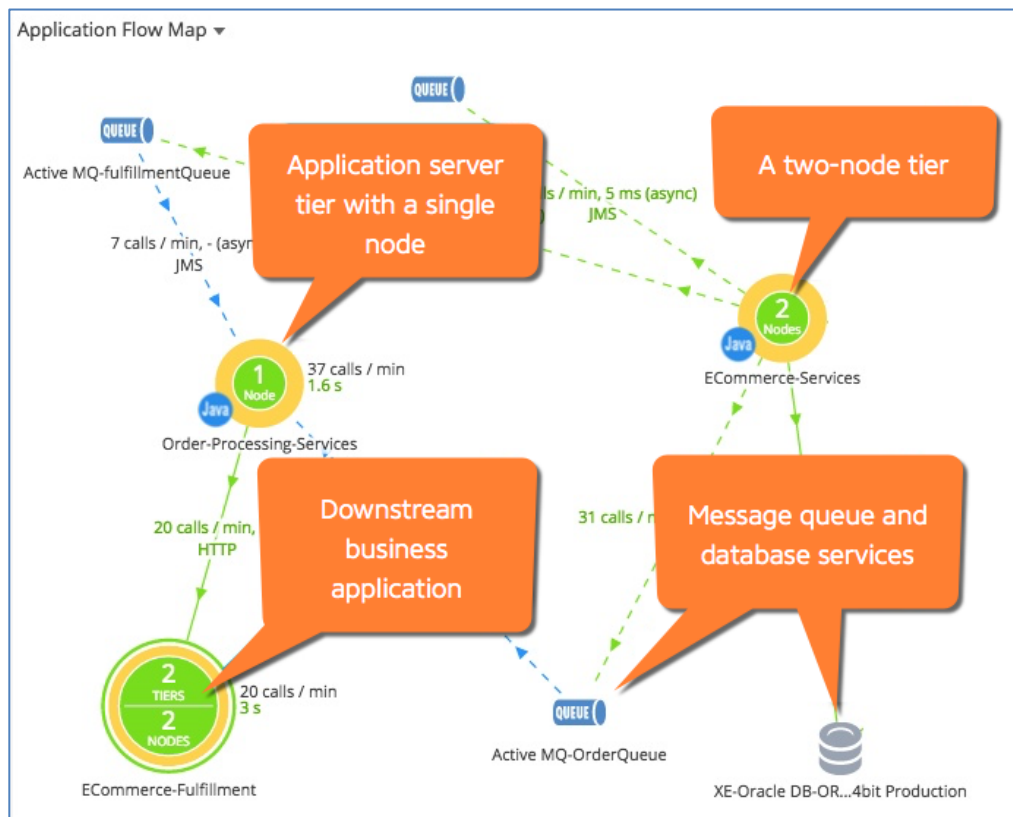
Application Agent	AppDynamics application agent shall be installed on target Application server host and deploy to those Application Server instance. The collected data shall be sent to the Controller and Event Service. AppDynamics provide support to various programming languages including - Java - .Net - Node.js and more
AppDynamics EUM Mobile and Browser (JavaScript) Agent	AppDynamics EUM (End user monitoring) server is used to receive the collected data from the end user's Mobile App and Browser App. These data shall then be sent to AppDynamics Controller and Event Service. Browser (JavaScript)Agent usually is transparently injected into web applications through web server load balancer, for example F5 BigIP. When a user accesses the web page, the F5 load balancer injects JavaScript transparently. The JavaScript agent collects browser level performance data such as http request error, html and resource load time, JavaScript error, DOM ready time. The collected data shall be sent back to the EUM server in DMZ, EUM shall then consolidate them and send back to Controller and Event Service.

AppDynamics controllers support running on virtual machines and Unix based OS including Linux, AIX, HP-UX. The whole platform can be installed on-prem in a fully isolated network environment with no external connectivity needed. Administration and operations can be done through a web browser connected to the controller through an SSL connection. SSL communications can also be set up between different AppDynamics components. There is also a SaaS subscription offering.

2.3 Splunk AppDynamics Features

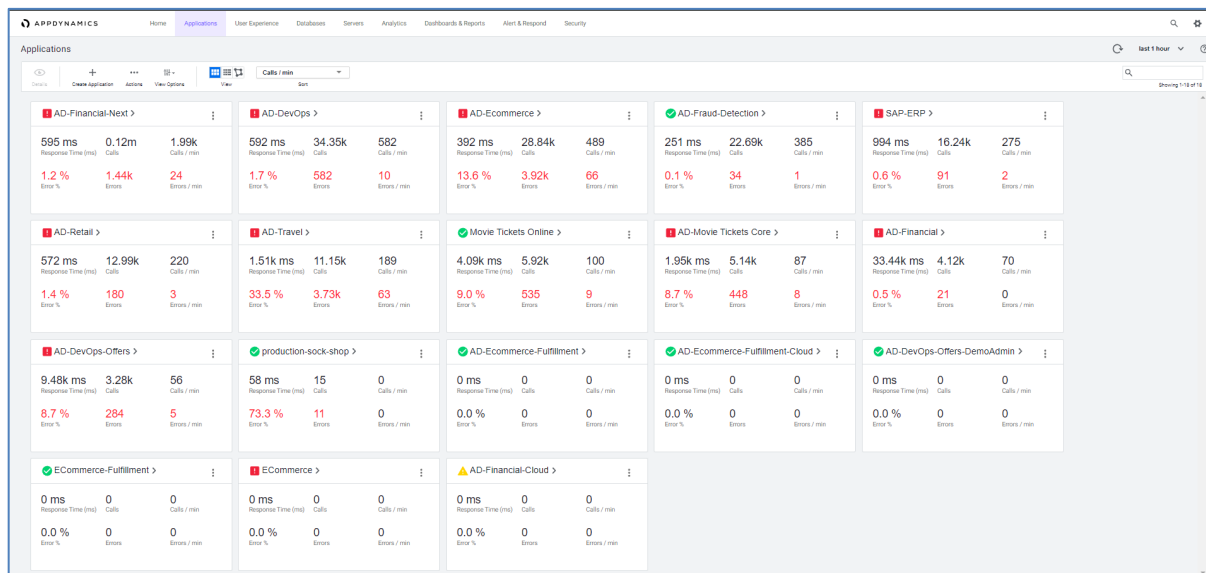
2.3.1 Flow Map

Flow Maps in AppDynamics are able to visualize all components in one view and their correlation with other services such as tiers, nodes, messages queues, databases in the environment, and business transactions. The auto-discovery feature in business transactions helps to correlate each business transaction and its service derivatives automatically and in real time.

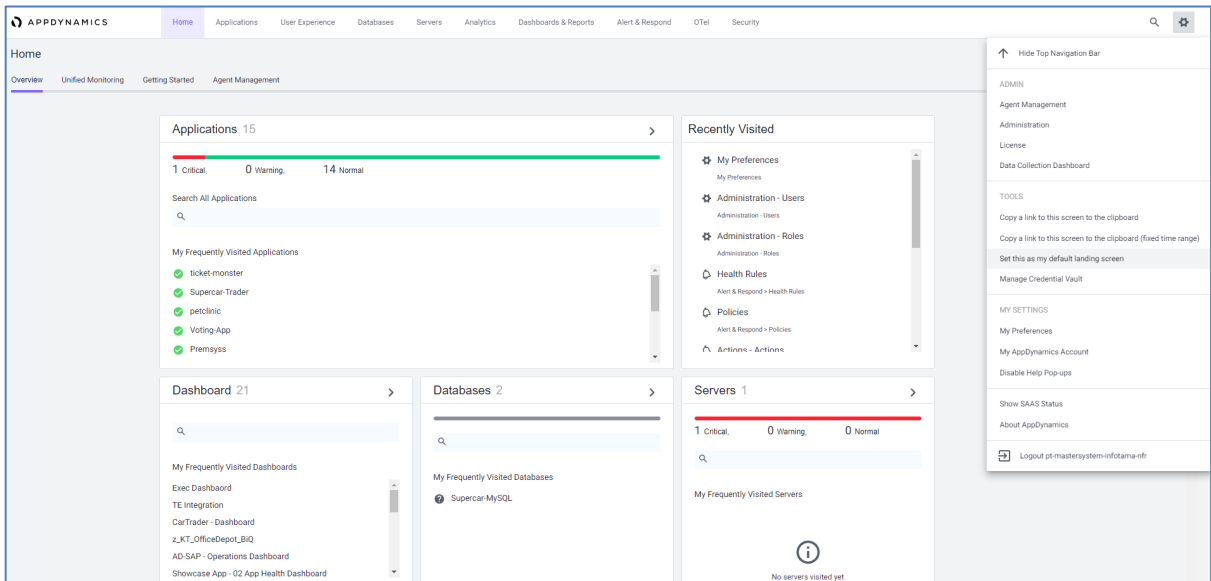
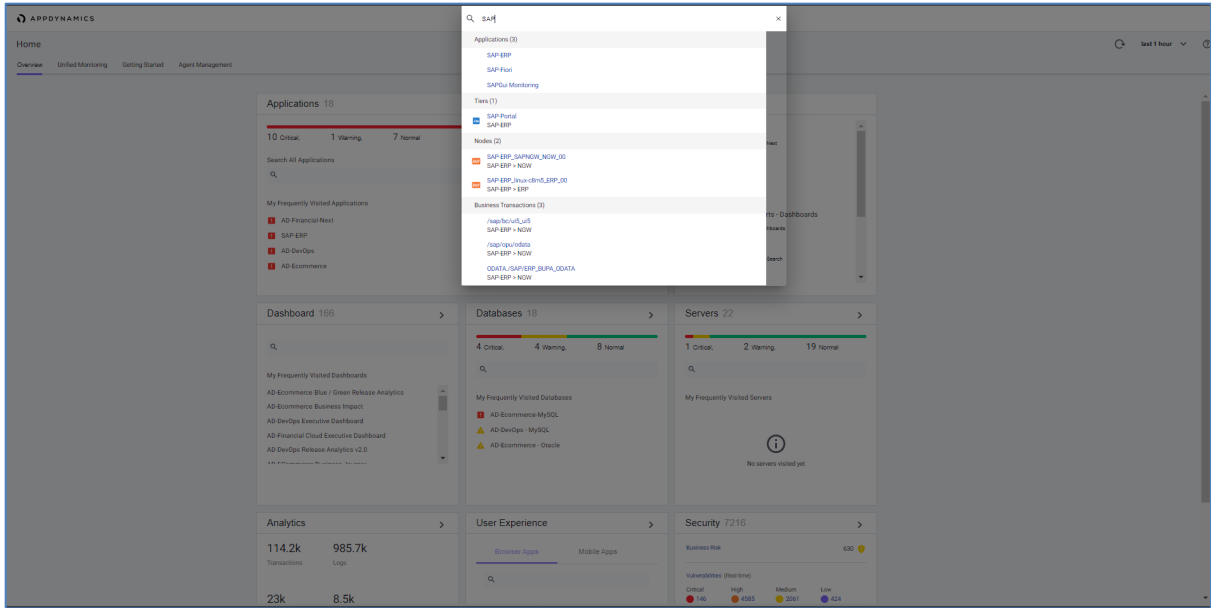


2.3.2 Homepage

AppDynamics Homepages can be accessed via a controller with port HTTP 8090 or HTTPS 8181. The initial display of the AppDynamics homepage will display an overview page. On the overview page there is health information for each application that is monitored with the status indicator red meaning critical, yellow meaning warning and green meaning normal. Then there is Recently Visited Information, Dashboard Information, Database, Server, Analytics, User Experience and Security which can be clicked to directly enter the menu.



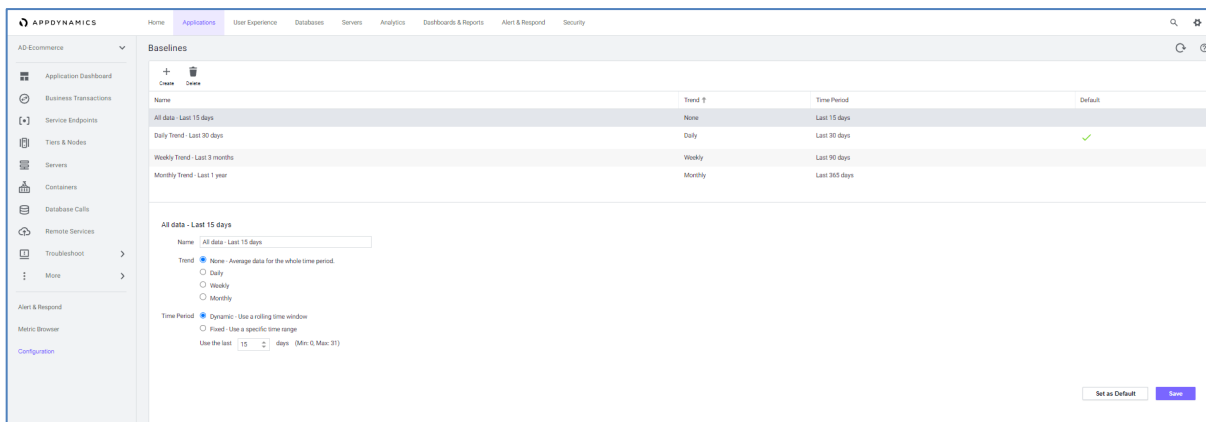
On the Application page there is health information for each application. In 1 application card there is information about Health status, Application Name, Average response time, total calls, Call/min, Percentage of errors, total errors and errors/mins. The view of the application can be displayed in the form of Cards View, Grid View or Flow View. In particular, in the Flow view you can see application correlations if they exist. Then, in the display there is a search bar which makes it easier for users to search for applications.



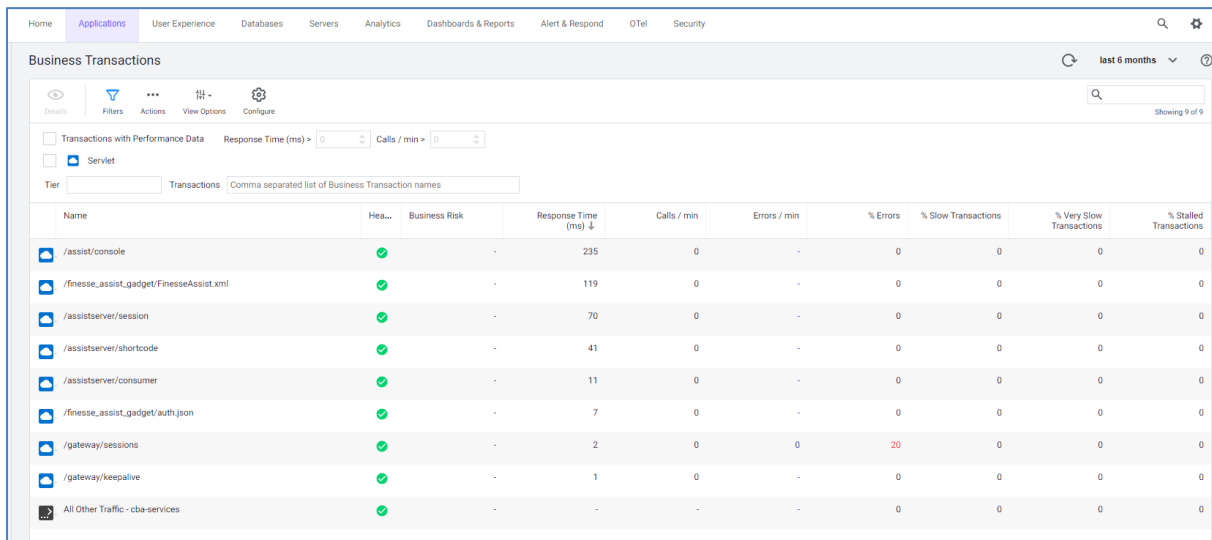
2.3.3 Application Monitoring



On the Application Monitoring page, there is a display of each application being monitored. Details of each application that is monitored include information on flow maps, correlation of dependencies for each service in the application, information regarding total calls, calls/min, average response time, total errors, errors/min, Health for Business transactions, Nodes, Servers and also current events. happens in the application. Then there is a Transaction Scorecard which informs the percentage of each transaction that is normal, slow, very slow, stalled and error.



The health status of each service can be customized using a baseline. The baseline in AppDynamics has 2 variables, namely Time Over which can calculate data based on considered data or seasonal baseline based on a predetermined season.



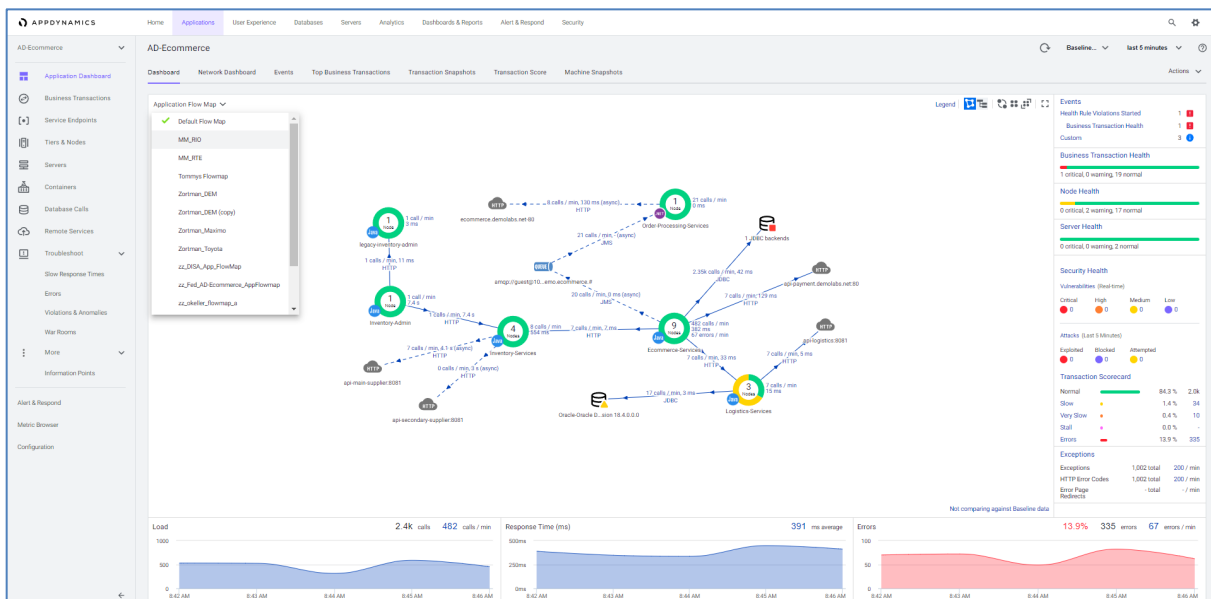
The screenshot shows the 'Business Transactions' page in AppDynamics. The page has a navigation bar with tabs: Home, Applications (selected), User Experience, Databases, Servers, Analytics, Dashboards & Reports, Alert & Respond, OTel, and Security. Below the navigation bar, there's a sub-header 'Business Transactions' with a refresh icon, a time range selector set to 'last 6 months', and a help icon. The main content area has tabs: Details (selected), Filters, Actions, View Options, and Configure. Below these tabs, there are checkboxes for 'Transactions with Performance Data' and 'Servlet'. There are also input fields for 'Response Time (ms)' and 'Calls / min'. A 'Tier' dropdown is set to 'Transactions', and a text input field contains 'Comma separated list of Business Transaction names'. The main table displays transaction metrics. The table has columns: Name, Health (green checkmark), Business Risk (minus sign), Response Time (ms) (down arrow), Calls / min, Errors / min, % Errors, % Slow Transactions, % Very Slow Transactions, and % Stalled Transactions. The table lists 9 transactions, including /assist/console, /finesse_assist_gadget/FinesseAssist.xml, /assistserver/session, /assistserver/shortcode, /assistserver/consumer, /finesse_assist_gadget/auth.json, /gateway/sessions, /gateway/keepalive, and All Other Traffic - cba-services. The /gateway/sessions transaction shows a % Errors of 20, which is highlighted in red.

Name	Health	Business Risk	Response Time (ms)	Calls / min	Errors / min	% Errors	% Slow Transactions	% Very Slow Transactions	% Stalled Transactions
/assist/console	✓	-	235	0	-	0	0	0	0
/finesse_assist_gadget/FinesseAssist.xml	✓	-	119	0	-	0	0	0	0
/assistserver/session	✓	-	70	0	-	0	0	0	0
/assistserver/shortcode	✓	-	41	0	-	0	0	0	0
/assistserver/consumer	✓	-	11	0	-	0	0	0	0
/finesse_assist_gadget/auth.json	✓	-	7	0	-	0	0	0	0
/gateway/sessions	✓	-	2	0	0	20	0	0	0
/gateway/keepalive	✓	-	1	0	-	0	0	0	0
All Other Traffic - cba-services	✓	-	-	-	-	0	0	0	0

Sorting and filtering can be done by clicking on the title of the information you want to sort, such as sorting by summary, Exception/min, Transaction and others.



The Application Monitoring display in AppDynamics is also equipped with Time Selection. Time Selection functions to determine the time period you want to see in an application. Time Range itself automatically displays flow maps or any data according to the time range that has been selected.

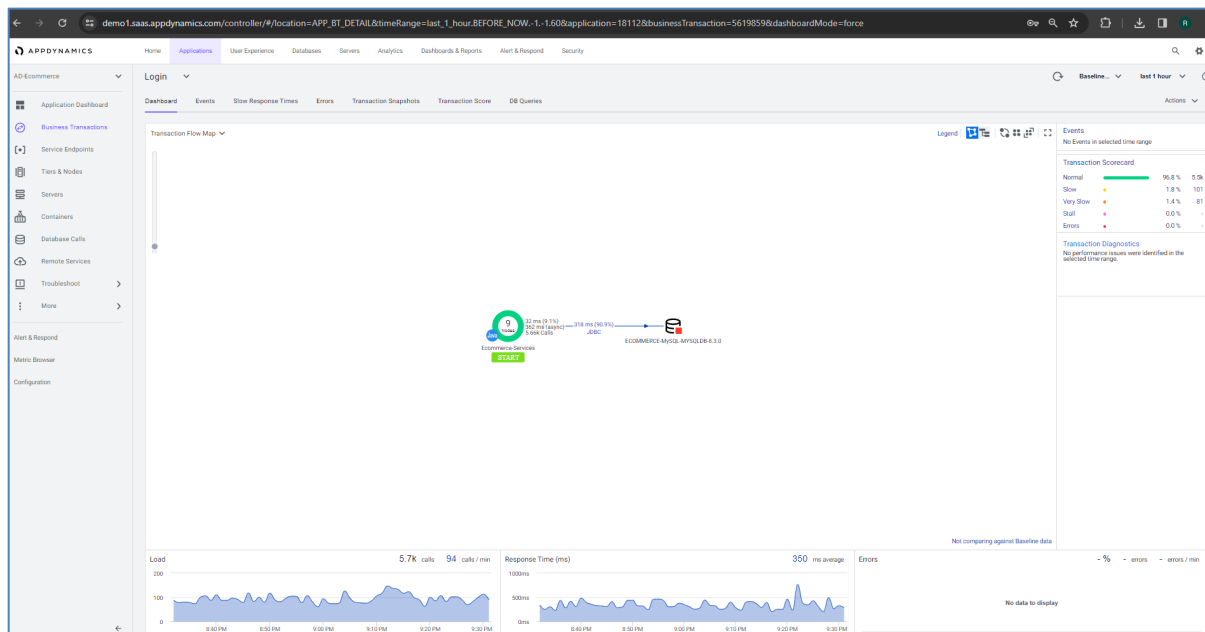


Application Flow Maps in AppDynamics can also be customized. Customization aims to enable users to carry out monitoring according to their individual needs. Services on flow maps can be removed or displayed so that users can focus on the services they want to monitor.

2.3.4 Business Transaction

Business transactions can correlate user transactions end-to-end. For example, for applications that require a "user login" login, then carrying out a "transfer" of conformation can be identified as a business transaction. Each business transaction will be grouped based on the transaction name. Each business transaction has a baseline for response time, calls/min, and error rate which is automatically formed using baseline dynamics. AppDynamics monitors all business transaction metrics such as response time, error rate and calls per minute.

For each business transaction, there is a flow map and also the performance of call/min, response time, error rate and transaction scorecard which can be an indication of whether each incoming transaction is normal, slow, very slow, stalled or has an error. Figure XX shows an example of a flow map of a login transaction and on the right there are details of the entire transaction which shows that there are no errors in the transaction.



Specific information from transactions such as customer name, customer type, total rupiah and others can be retrieved using a data collector. Data collectors are used to complement business

transactions and transaction analytics data with application data. Application data provides context for business transactions, for example a user wants to display all customers who use different transfer types who have a bad user experience.

If there is a bad experience with a business transaction, you can see the details of the transaction using transaction snapshot.

Login

Dashboard

Events

Slow Response Times

Errors

Transaction Snapshots

Transaction Score

DB Queries

All Snapshots

Transaction Diagnostics

Slow and Error Transactions

Diagnostic Sessions

Periodic Collection

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

Filter

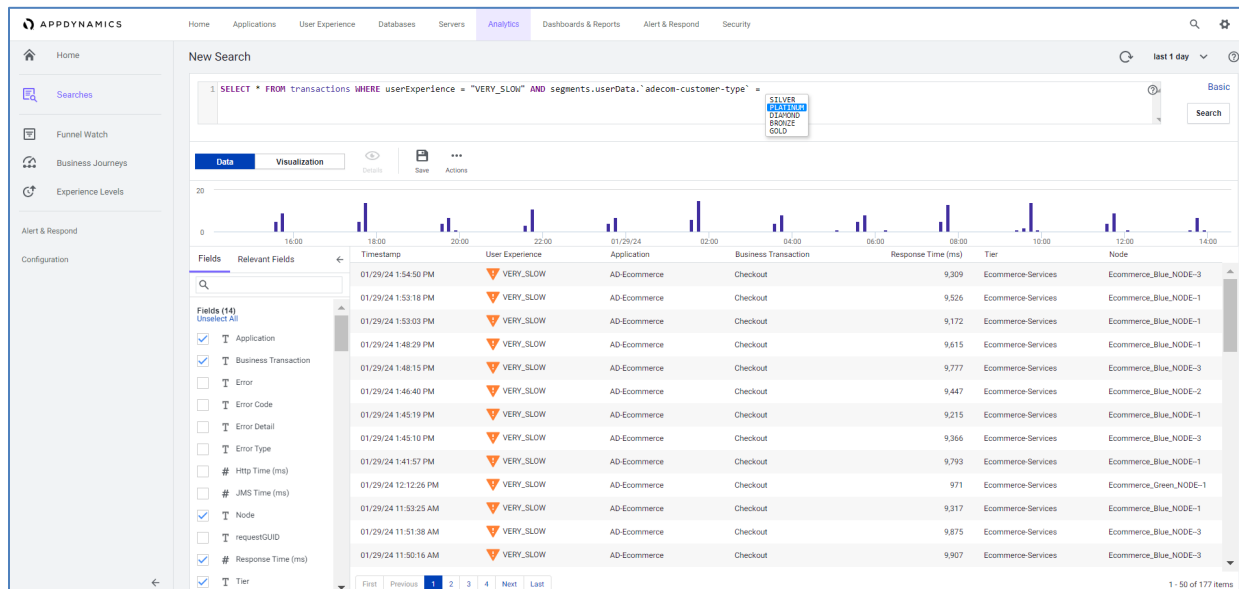
Filter

Filter

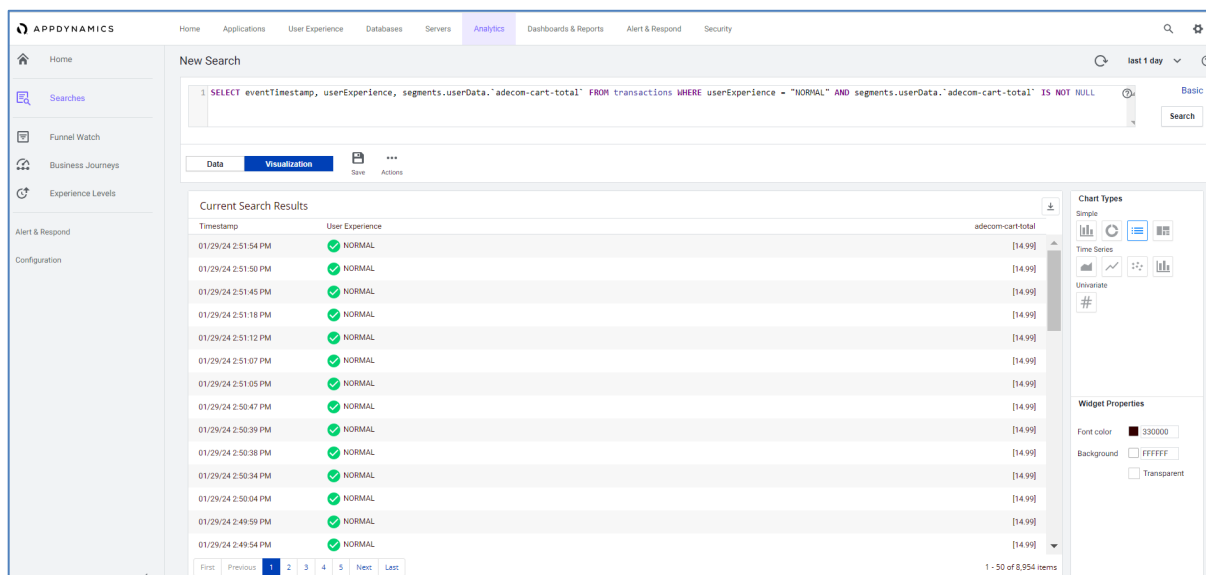
Detailed transaction snapshots can provide transaction tracing information ranging from overview, slow calls or errors, performance of calls to the database, server performance and network performance. As below, an error can be found that causes a login transaction.

Transaction: 535cedbd-2604-44d5-b6d1-052a27ea2217148 ms · WebFrontEnd	
OverviewSlow Calls & ErrorsDB & Remote Service CallsServerNetworkMore	
Summary	Slowest Calls and Errors
User Experience Error Execution Time 148 ms CPU Time Timestamp 01/29/24 8:43:21 PM (server) 01/29/24 8:43:21 PM (agent) Node WebFrontEnd-2 Tier WebFrontEnd Business Transaction Login URL (not found) More Details	No Slow Calls Found Error java.lang.IllegalMonitorStateException at java.util.concurrent.locks.ReentrantReadWriteLock\$Sync.tryRelease(ReentrantReadWriteLock.java:371) at java.util.concurrent.locks.AbstractQueuedSynchronizer.release(AbstractQueuedSynchronizer.java:1261) at java.util.concurrent.locks.ReentrantReadWriteLock\$WriteLock.unlock(ReentrantReadWriteLock.java:1131) at com.appdynamics.sample.service.BackgroundWorker\$Worker.run(BackgroundWorker.java:59) at java.lang.Thread.run(Thread.java:750)

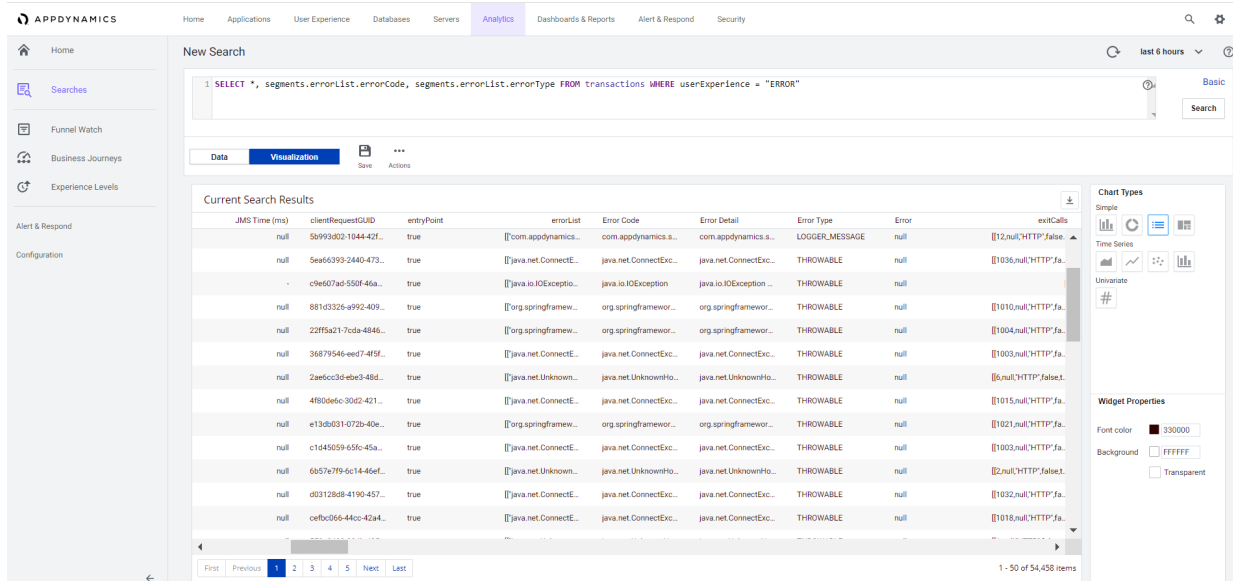
Following figure shows an example of carrying out a business transaction breakdown using the context of application data, in this case a platinum type customer who has a bad user experience.



This shows an example of the correlation between transaction analytics data and the performance of business transactions. Next figure shows a query for each normal transaction and displays the total cart data.



Following figure shows the analytics display with each transaction that experienced an error with detailed error code, error list and error details.



Business Transaction Trend data can also be displayed on the home page in the analytics menu. On the home menu analytics page displays trends regarding number of transactions, top 10 transactions by volume, end user session summary and trend, browser session-bounce rate and also other information that can be customized to be displayed. Trend data can also be displayed according to the selected time range such as last 1 day, last hour, or custom range. In Figure 4.19 is the home display of the analytics page in AppDynamics.

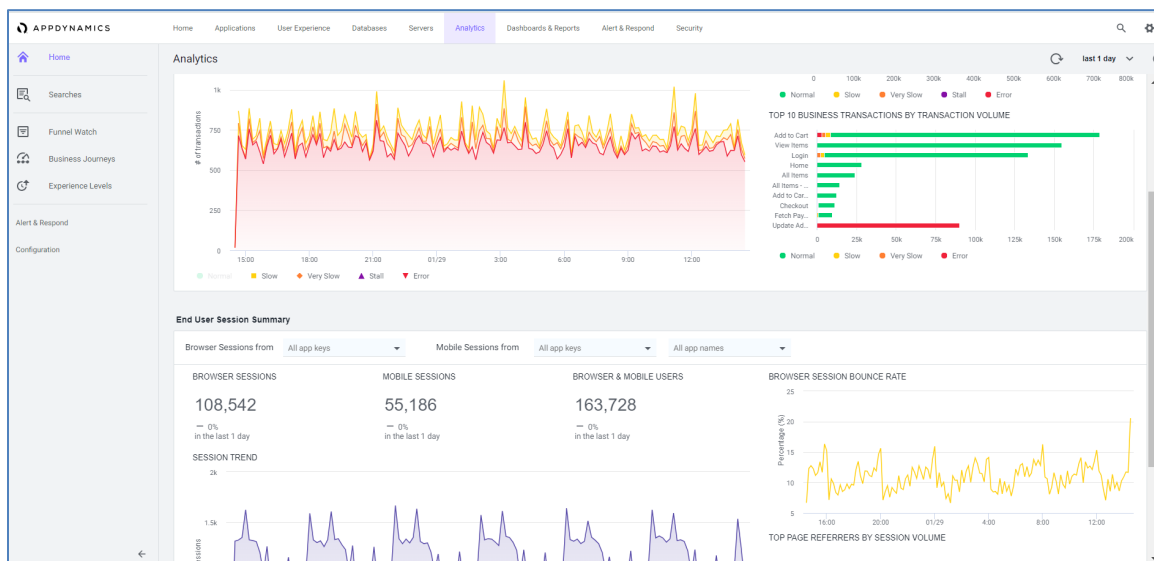


Figure 2.1 Home analytics Displays

Below figure shows the top 5 or top 10 views of transactions or services grouped based on load amount, response time, error rate, slow transactions and others.

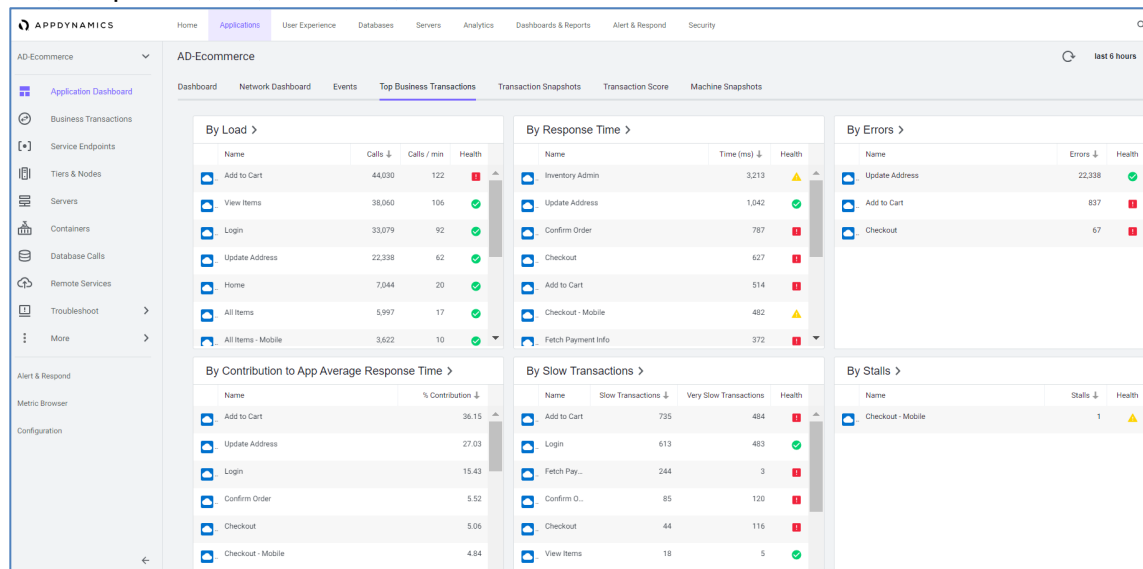


Figure 2.2 Top business transaction Display

AppDynamics Agent installer simplifies the deployment process by instrumenting applications quickly. Users can manage applications that will be instrumented with agent installers safely and easily in the application. The entire process of adding arguments to the application has been well documented and the procedures and guidelines can be carried out easily.

2.3.5 Deep Infrastructure Monitoring

Infrastructure Visibility in AppDynamics provides end-to-end visibility into the performance of the hardware running your applications. You can use Infrastructure visibility to identify and troubleshoot problems that can impact application performance such as Server Fault, JVM Crash, and Network Packet loss.

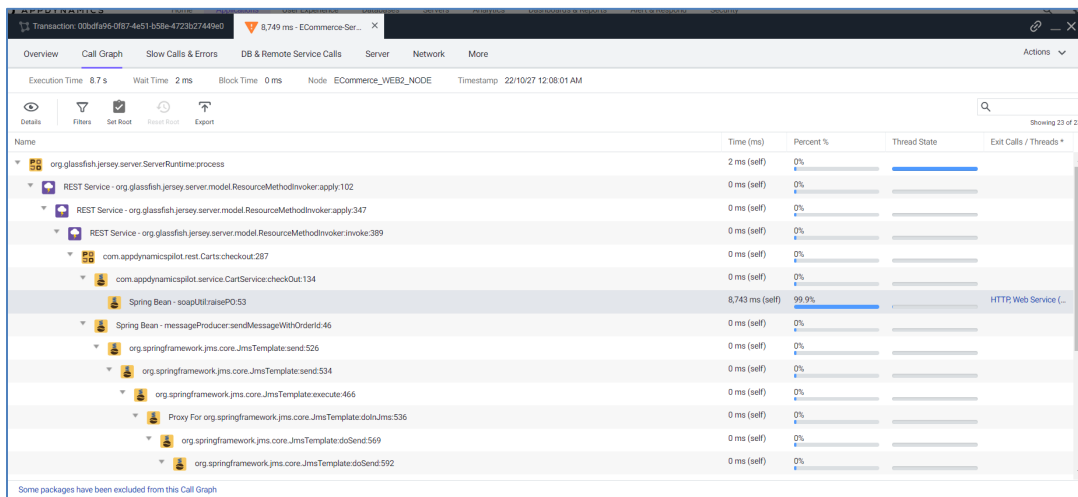
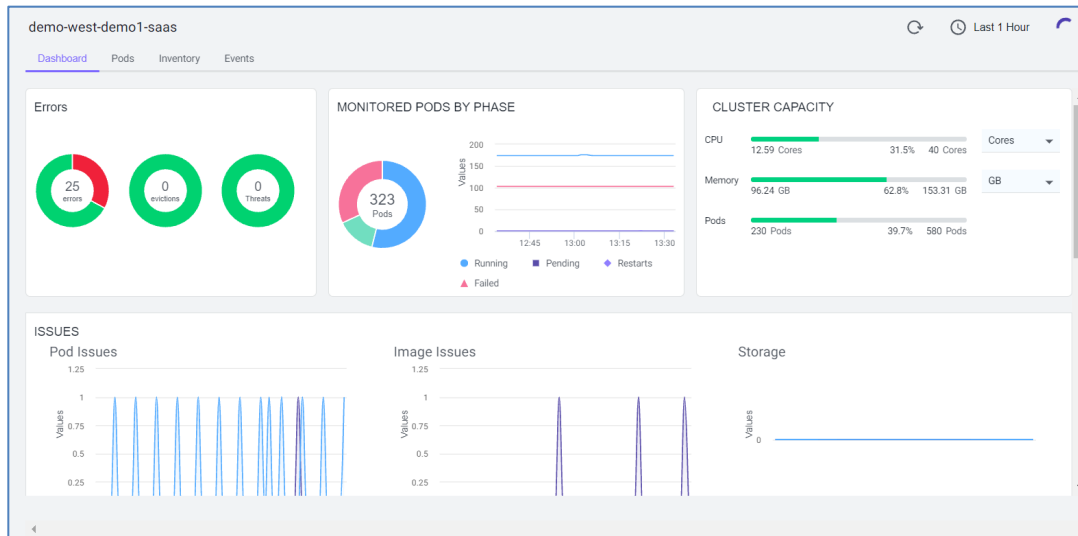
Infrastructure Visibility allows you to isolate, identify, and solve problems such as too much time spent in garbage collection, packet loss between 2 nodes that causes retransmissions and slow calls, inefficient processes that cause high CPU, too high read/write rates on a specific disk or partition. Infrastructure Visibility is based on the Machine Agent running with the App Server Agent on the same machine. Both agents provide multi-layer monitoring.

- App Server Agent collects application-related metrics and identifies applications, tiers, and nodes with slow transactions, stalled transactions, and performance issues in other applications.
- Network Agent monitors network packets sent or received at each node and identifies if there is loss, TCP Bottlenecks, high round trips, and other network issues.
- Machine Agent collects metrics such as local processes, services, resource utilization, metrics for disk utilization, memory, CPU, and network interfaces.

Multi-layer monitoring makes it possible to correlate problems with applications and services, processes, hardware, networks and other issues on the server.

	This Agent Monitors...	Example Metrics
1 App Agent 	apps app servers JVMs CLRs	Slow Transactions Stalled Transactions Response Times Wait Times Block Times Error %
2 Network Agent 	network packets TCP connections TCP sockets	Performance Impacting Events Packet loss and retransmissions Round Trip Times for data transfers TCP Window Size (Limited or Zero) Connection setup/teardown errors
3 Machine Agent 	Server Visibility processes services caching swapping paging queueing	Hardware/Software Interrupts Virtual memory/swapping Process faults CPU/disk/memory utilization by process
	Hardware/OS disks volumes partitions memory CPU network interfaces	CPU busy times Memory utilization Disk reads/writes JVM crashes

AppDynamics also provides easy visibility into container-based applications. The container dashboard in AppDynamics displays the resource utilization of the application container and the underlying host machine. The Container Dashboard can be accessed from the Server tab in the Controller UI. The Containers page lists all monitored Containers used by applications registered to the Controller.



CallGraph helps to carry out deep tracing down to code-level and service-level, as well as we can get insight into what percentage each service contributes to the transaction. AppDynamics Call Graph is also able to provide correlation of slow/very slow/stall transactions with infrastructure and network metrics. This provides insight into whether the infrastructure and network have an impact on the user's experience of the transaction.

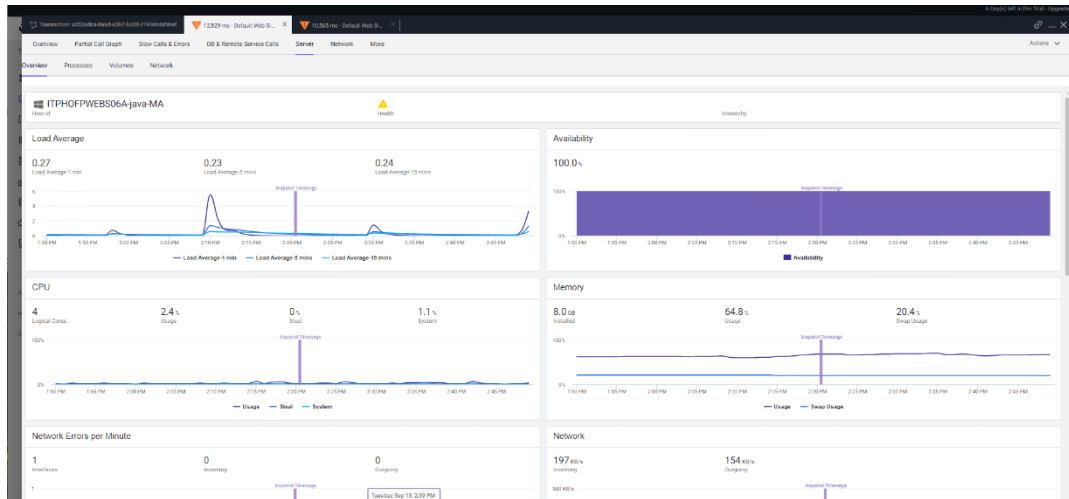


Figure 2.3 Call Graph AppDynamics Correlation

Drilldown analysis of slow transactions to the infrastructure shows that there is a disk space warning because 20% remains.

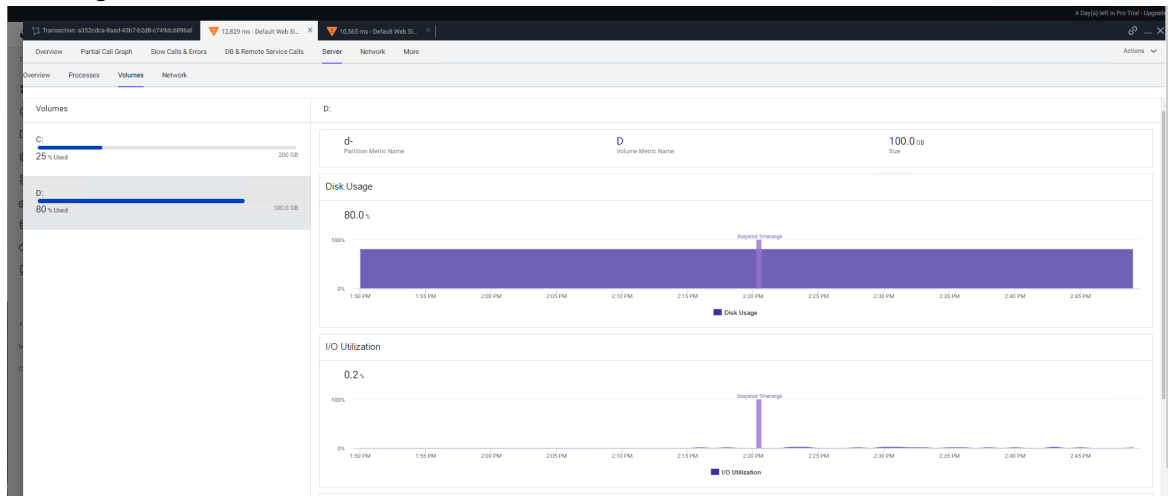


Figure 2.4 Performance AppDynamics Infrastructure correlation

Database Monitoring

Database visibility in AppDynamics provides end-to-end visibility into database performance. Database Visibility helps troubleshoot problems such as slow response time and excessive load. Database Visibility also collects metrics from database activities such as:

1. SQL Statements or stored procedures that use the most system resources
2. Statistics on procedures, SQL Statements, and SQL Queries
3. Long time fetching, sorting, or waiting
4. Activity from the previous day, week, or month

AppDynamics Database Agent is a standard java program that conducts a collection of performance Metrics about database instances and database servers. The following is the Database Agent topology.

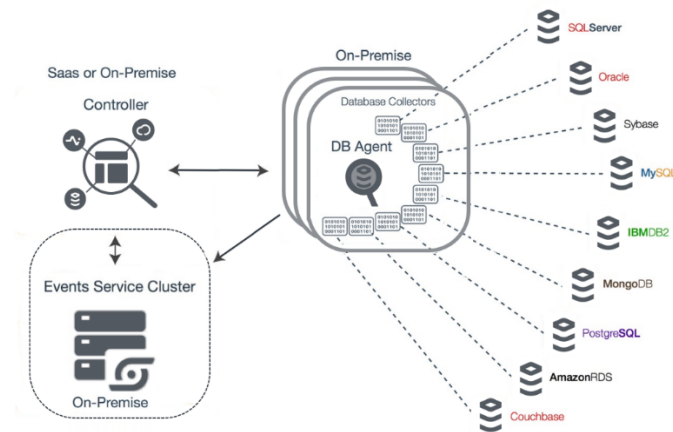


Figure 2.5 Topology of Database Agent

Table 2.1 Database Supported Environment

Database	Supported Through Amazon RDS	Version
Apache Cassandra		>= 3.11.4
Datastax Enterprise (DSE) Cassandra		5.1, >= 6.7.3
Couchbase		>= 4.5
IBM DB2 LUW		9.x, 10.x, 11.x
MongoDB, MongoDB cluster		>= 2.6
MySQL	Yes	All versions including MySQL Version 8.0, Percona, MariaDB, and Aurora
Microsoft SQL Server	Yes	2005, 2008, 2012, 2014, 2016, 2017, 2019, 2022, and SQL Azure
Microsoft SQL Server on Linux		SQL Server on Linux is currently available as a public preview and is not recommended for production use.
		Database Visibility works well with this preview release, but monitoring results may vary until a stable version of SQL Server on Linux is available.
Oracle, Oracle RAC	Yes	10g (>= 10.2), 11g, 12c, 18c, and 19c
PostgreSQL	Yes	All the versions including Azure Database for PostgreSQL and Aurora
Sybase ASE		>= 15
Sybase IQ		All versions
Hadoop		Monitored using appdynamics extension

Network Monitoring

Network Visibility in AppDynamics is able to expand AppDynamics intelligence from the application stack to the network. Connectivity between each service component is also depicted in the Network Dashboard. Network Visibility can help reduce or eliminate guesswork when identifying root cause analysis. Network Agent and App Agent work together to automate the mapping of TCP connections to the application flows that use

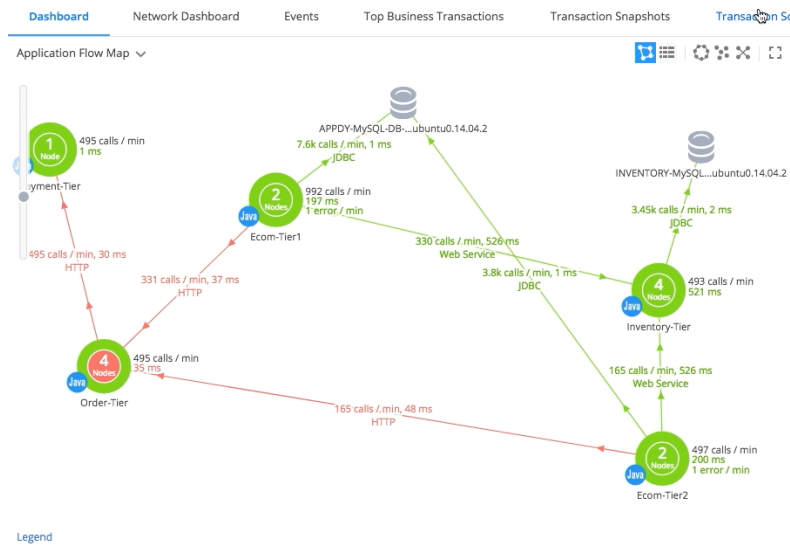


Figure 2.6 FlowMaps Network AppDynamics display

Analytics

Analytics can be used for APM, RUM Browser, Mobile RUM and Synthetic Browser modules. Analytics itself is able to extract data from the parameters you want to take, such as the number of unique users, coverage rate, revenue, total cart, customer name, customer type and so on. Each analytics data also has a baseline so you can see the correlation between business metrics and application metrics. Analytics is also used to see trends in each metric such as business transaction trends and others.

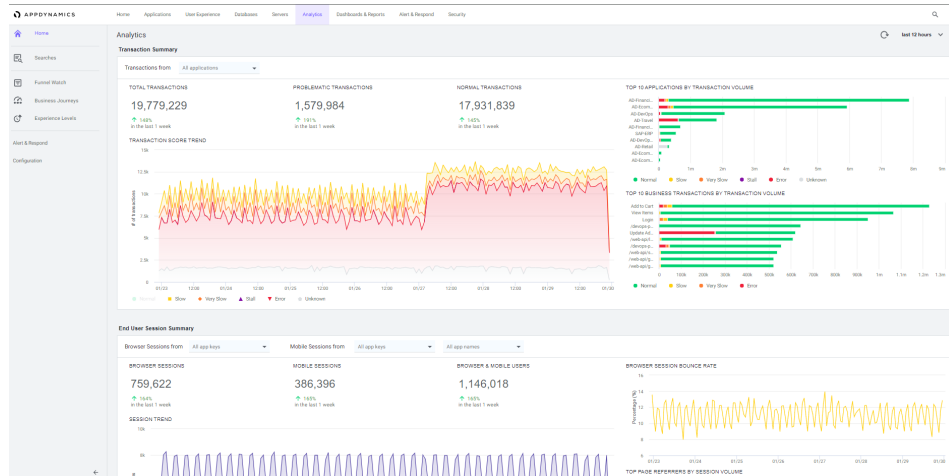


Figure 2.7 Analytics AppDynamics

Figure 4.42 shows an example of a custom dashboard that uses several analytics widgets, so that in 1 dashboard there is business data such as unique sessions, percent-age offer given, conversion rate, revenue and also a conversion funnel which shows the journey from a user until it turns into a conversion. rate along with the performance status of each module.

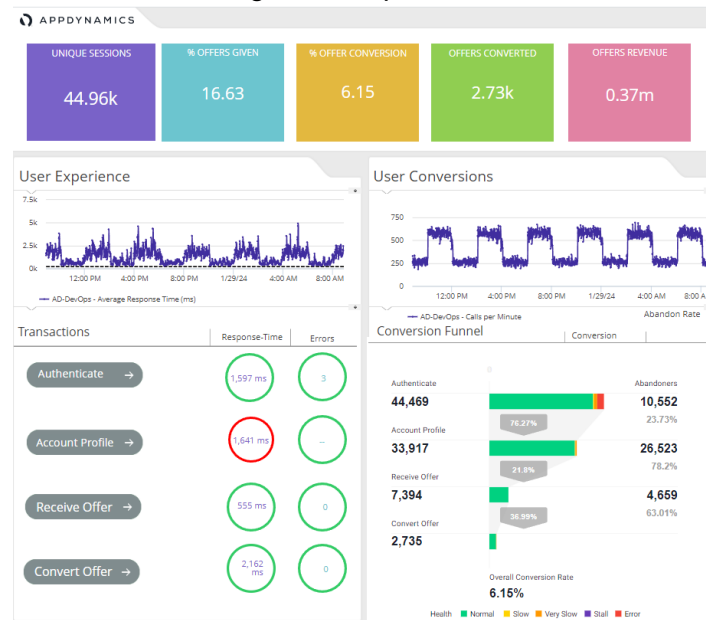


Figure 2.8 Custom Dashboard AppDynamics display

Custom Dashboards can be done in AppDynamics to display the data information needed to be monitored such as total unique sessions, conversion rate percentage, total revenue, transaction journey and service.

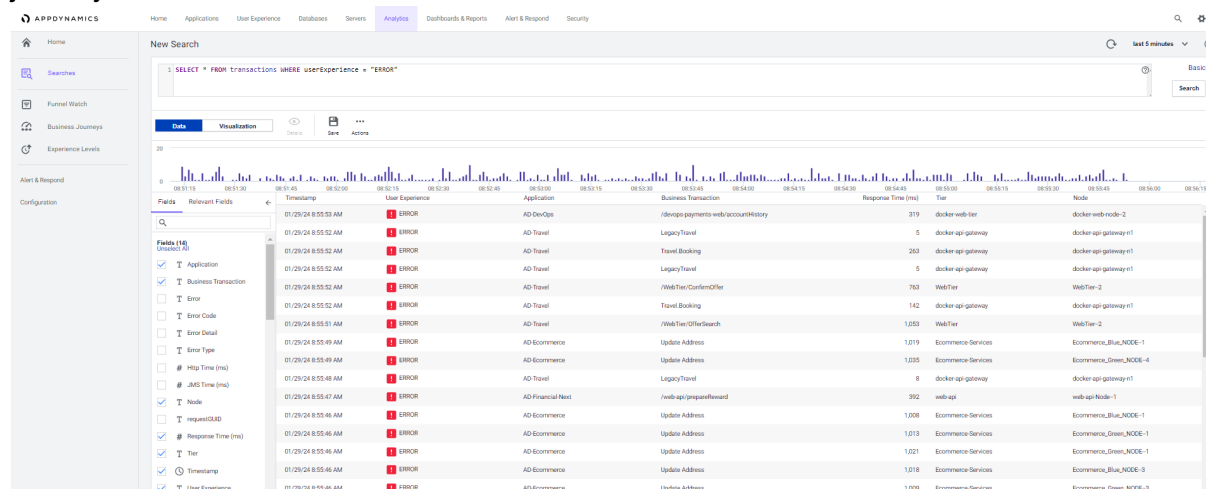


Figure 2.9 Feature ADQL

AppDynamics also has an ADQL (AppDynamics Query Language) feature which allows users to query data according to the data they want to display. Every search result in ADQL can be directly entered into a custom dashboard.

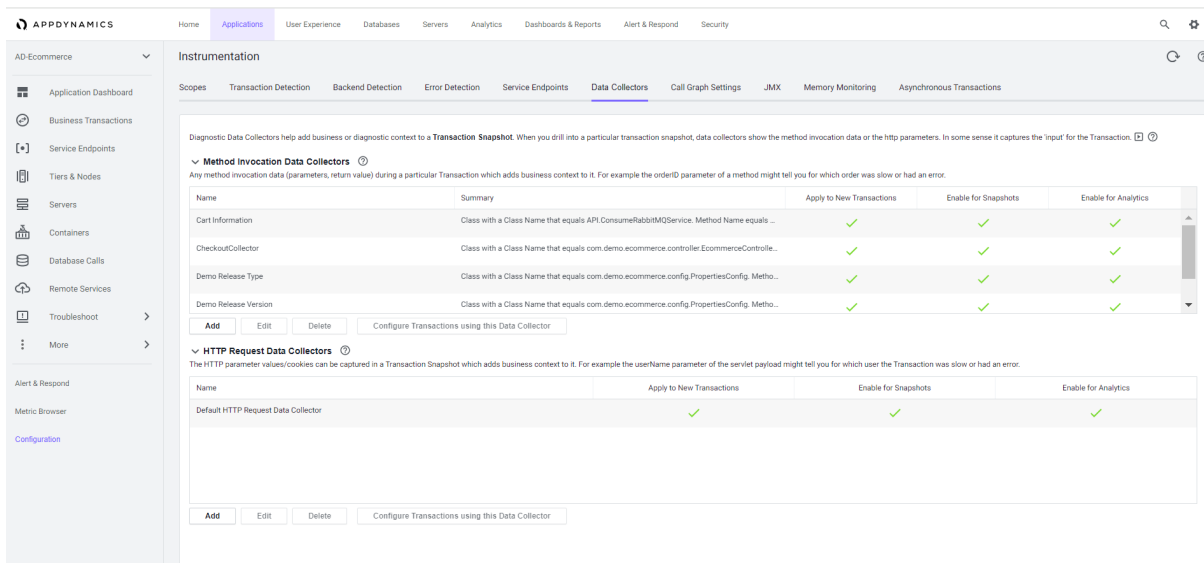


Figure 2.10 Data Collector Configuration menu

Figure 4.44 shows a configuration image for the data collector. Where the data collector is used to capture variable data based on method invocation or http request. The data collector will later capture every data that has been configured for each transaction. The results from the data collector can be seen as in Figure 4.45.

Transaction: Mod972-4d2-4d4-9d4e-972d812289			
Overview Slow Calls and Errors Waterfall View Segment List Data Collectors			
Actions			
Name	Value	Tier	Node
ReleaseType	[blue]	Inventory Services	Inventory_Blu_NODE-4
Release Version	[1.1.2]	Inventory Services	Inventory_Blu_NODE-4
numberOfItems	[1]	Order Processing Services	Ecommerce_MLS_NODE
costOfItems	[14.99]	Order Processing Services	Ecommerce_MLS_NODE
contentLength	[965]	Order Processing Services	Ecommerce_MLS_NODE
time	[01/29/2024 16:21:32]	Order Processing Services	Ecommerce_MLS_NODE
content	[{"type":"Checkout","networkStatus":"Pass","items":[{"id":"BOOK TECH 1000","category":["INF","usd"],"sku":"4d87f95bc9e11ee-ad35-5e412d432218a","name":["Books"],"name":["Dash Course in Python"],"description":...	Order Processing Services	Ecommerce_MLS_NODE
orderContent	[API DTOs:MessageItemDTO]	Order Processing Services	Ecommerce_MLS_NODE
adecom-quantity	[1]	Ecommerce Services	Ecommerce_Blu_NODE-1
ReleaseType	[blue]	Ecommerce Services	Ecommerce_Blu_NODE-1
Release Version	[1.1.2]	Ecommerce Services	Ecommerce_Blu_NODE-1
adecom-customer-name	[Solomon]	Ecommerce Services	Ecommerce_Blu_NODE-1
adecom-customer-city	[San Francisco]	Ecommerce Services	Ecommerce_Blu_NODE-1
adecom-orderId	[b6e5a236a89071-d4d7-4d6d-4d40-4d4d4d4d4d4d]	Ecommerce Services	Ecommerce_Blu_NODE-1
adecom-cart-total	[14.99]	Ecommerce Services	Ecommerce_Blu_NODE-1
adecom-customer-type	[SILVER]	Ecommerce Services	Ecommerce_Blu_NODE-1
adecom-top-item	[Dash Course in Python]	Ecommerce Services	Ecommerce_Blu_NODE-1

Figure 2.11 Results of capturing collector data on transactions

Every metric that has been collected by the data collector can be displayed and queried in analytics. The example in Figure 4.46 displays query results using location parameters with specific locations. The output of this query is all transactions that have been filtered based on the location queried.

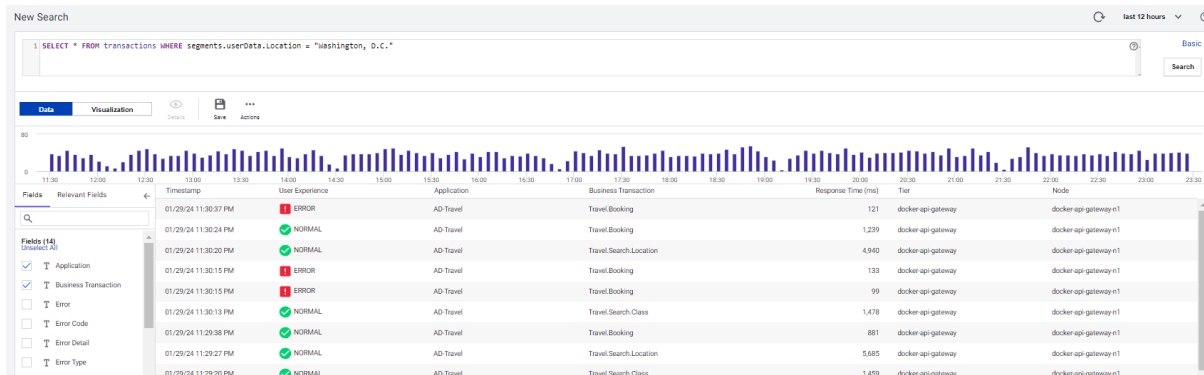


Figure 2.12 Query using variable metrics from data collector

Log Analytics

AppDynamics log analytics collects, correlates, and analyzes data to gain comprehensive insight into operational performance in real time. Some of the advantages of using AppDynamics Log Analytics are as follows.

- Collect real time machine data and insights
- Run Advanced Searches to find errors and perform log analysis
- Gain comprehensive operational intelligence
- Visualizing data on custom dashboards creates compelling insights
- Take action based on the right data by getting important

Error Detection

Errors in AppDynamics are divided into 2, namely Transaction Errors and Exceptions. Transaction errors are errors caused by unhandled errors, exceptions occurring in exit calls, HTTP error responses, or errors that are configured and customized to be displayed when the criteria that have been created are met. Meanwhile, an exception is every exception that is logged using log4j, log4net, or other logs. HTTP errors that are not found in business transactions, or errors when redirecting pages.

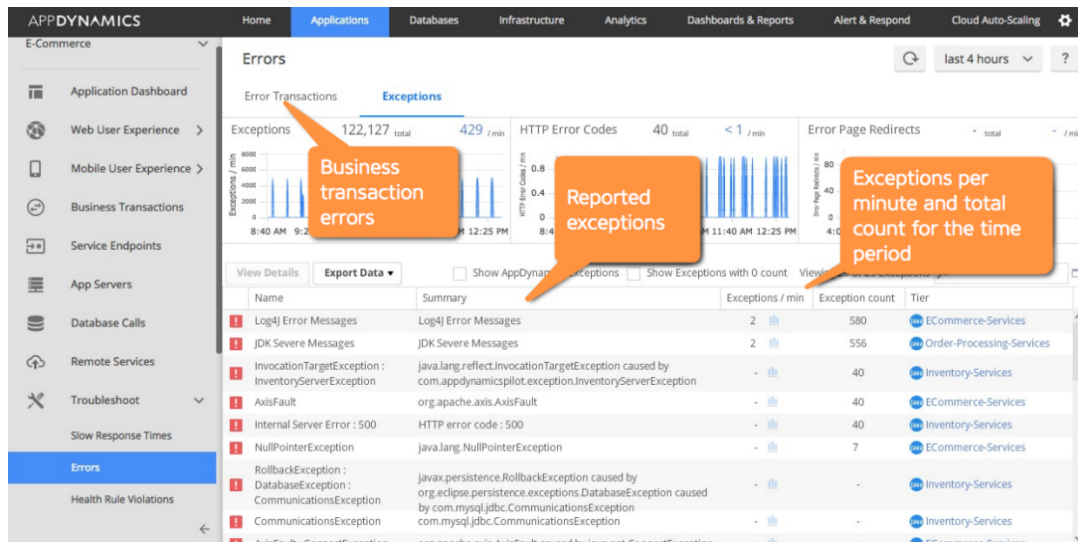


Figure 2.13 Error Detection AppDynamics

AppDynamics will automatically capture if an error transaction occurs. In Figure 4.48 you can see that there are transaction errors and AppDynamics presents the flow of transactions that occur and their dependencies and provides information on where the error occurred, as well as the error code and error messages.

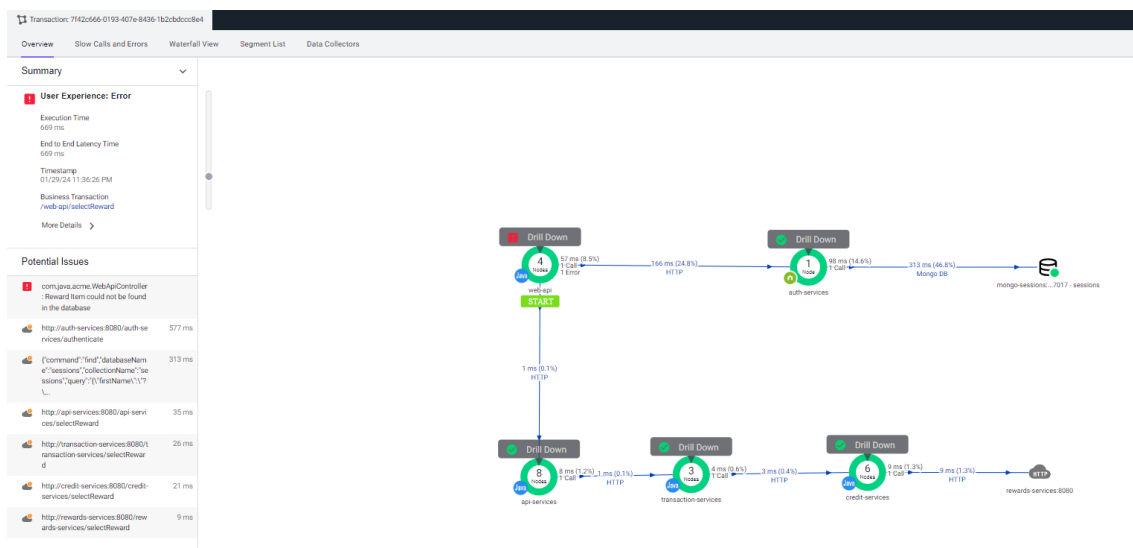


Figure 2.14 Root Cause Error from flow transaction display

Health Rule

Health rules in AppDynamics function to determine the criteria for a metric or transaction. Health rules themselves can determine whether a transaction or metric has normal, warning or critical status. Health rules can be set for each metric as needed, as in the example in Figure 4.49.

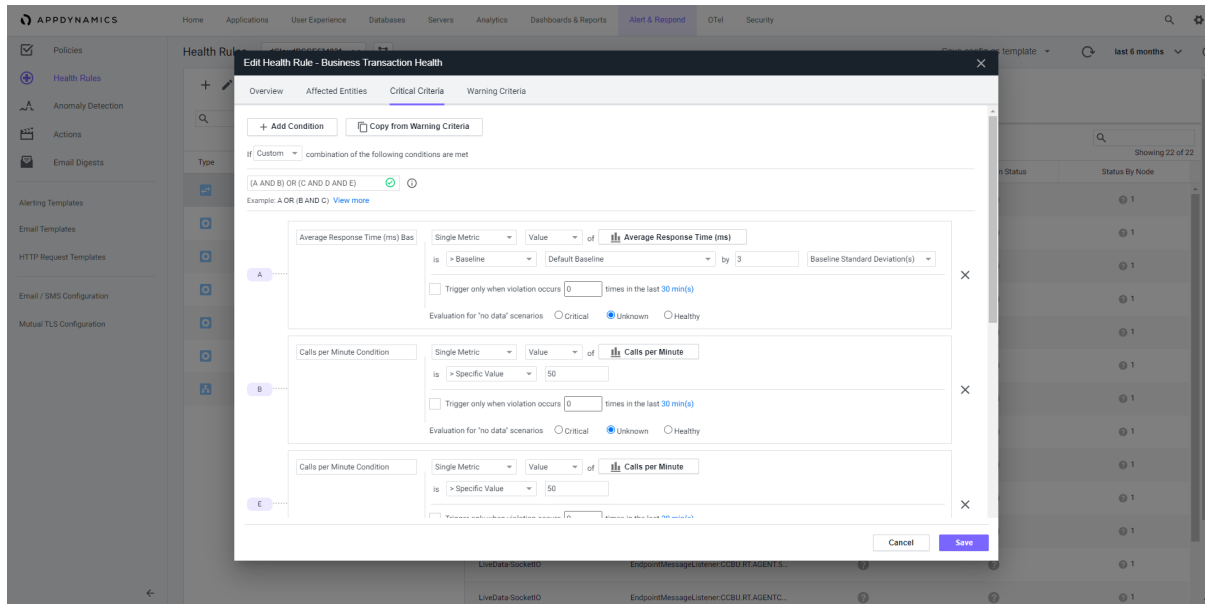


Figure 2.15 Health rule configuration menu

User Management

User Management in AppDynamics functions to manage user accounts, Single Sign-On (SSO) configuration, Secure Access Management. For Authentication and User management can be integrated using.

- Okta
- Onelogin
- Ping Identity
- Microsoft Active Directory
- Azure Active Directory
- IBM Cloud Identity
- Active Directory Federation Service (AD FS)

Administration in AppDynamics allows admins to add, edit or remove users, integrate with the LDAP system, create groups based on specific privileges or roles, and also create privileges for accessing the dashboard. Example image 4.50 shows the administration menu for the user.

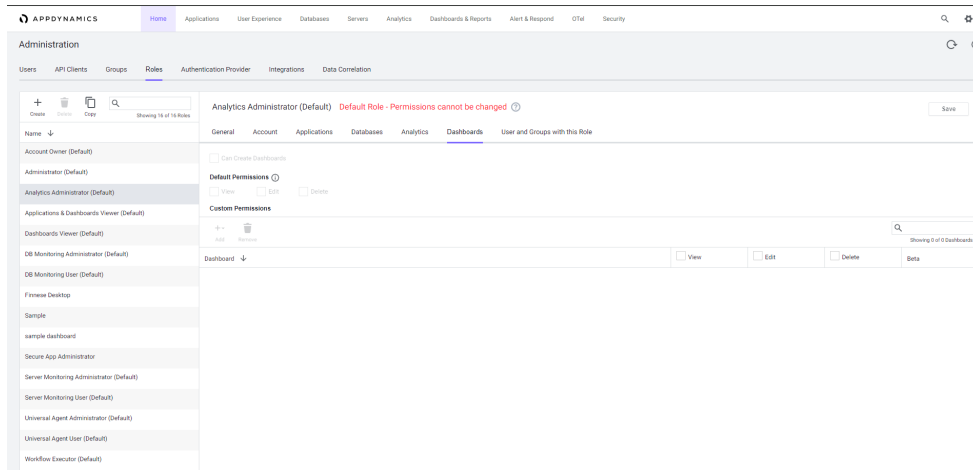


Figure 2.16 User configuration menu display

Specific user preferences can be made in my preferences menu. In this menu the user can change the date format, time zone and time format such as using 24-hour timing or not. Figure 4.51 shows my preferences menu.

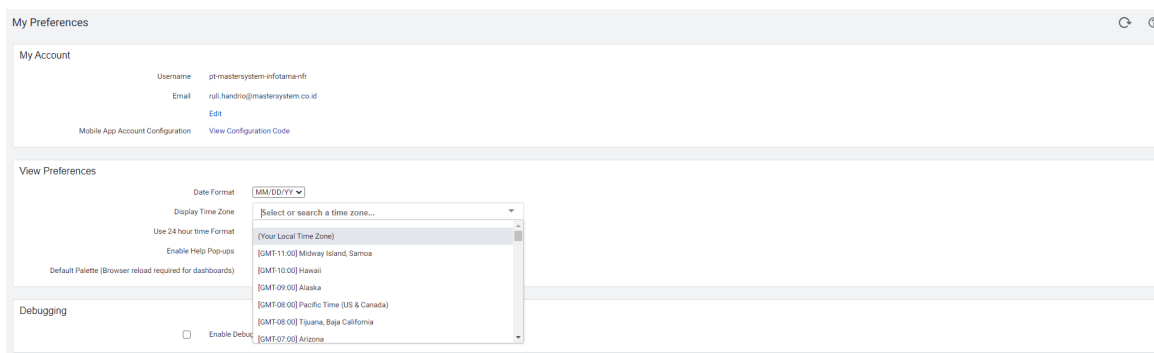


Figure 2.17 Display Time preferences menu

Dynamics Baseline

AppDynamics has a dynamic baseline feature which functions to carry out calculations for each metric to be able to determine a baseline based on hourly data. Baselines can also be used to carry out anomaly detection and capacity planning.

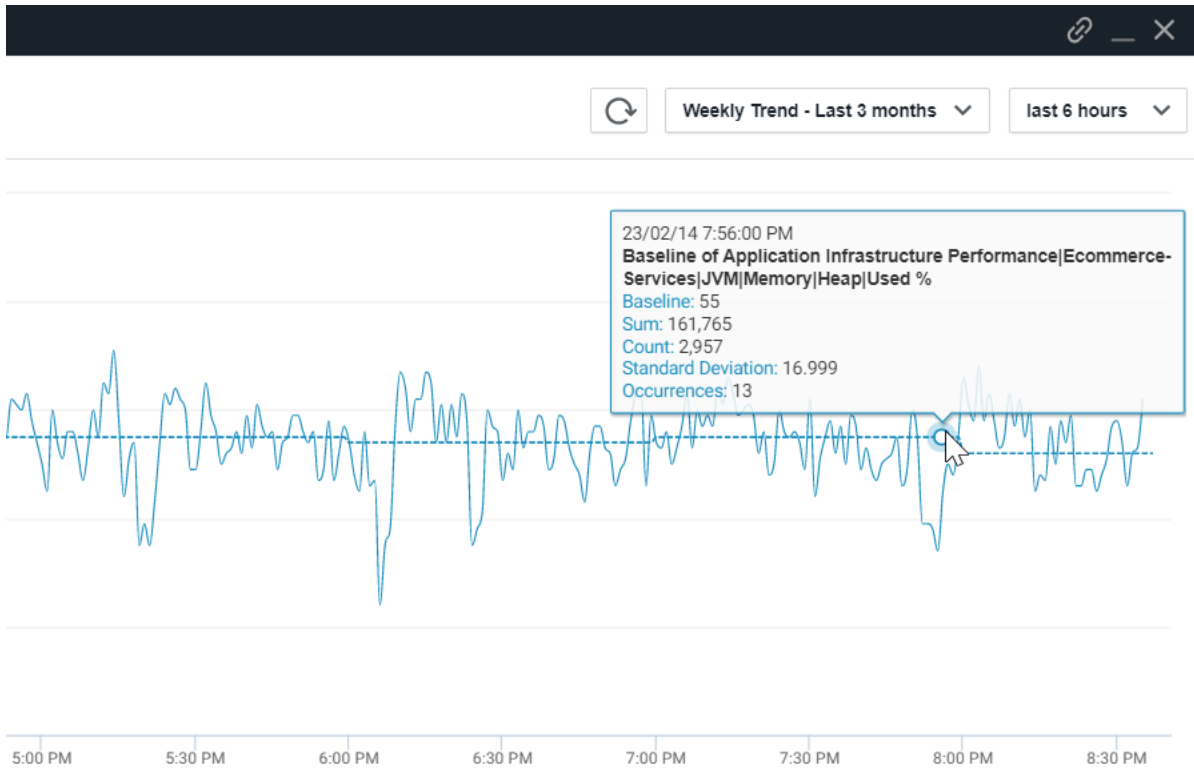


Figure 2.18 Dynamics baseline

The baseline in AppDynamics can also be used as a reference for future data projections, so that the dynamics baseline reference can be used as an alert and notification if the data is outside the baseline. For example, in Figure XX, the baseline has a high number, but the actual data is below the baseline.

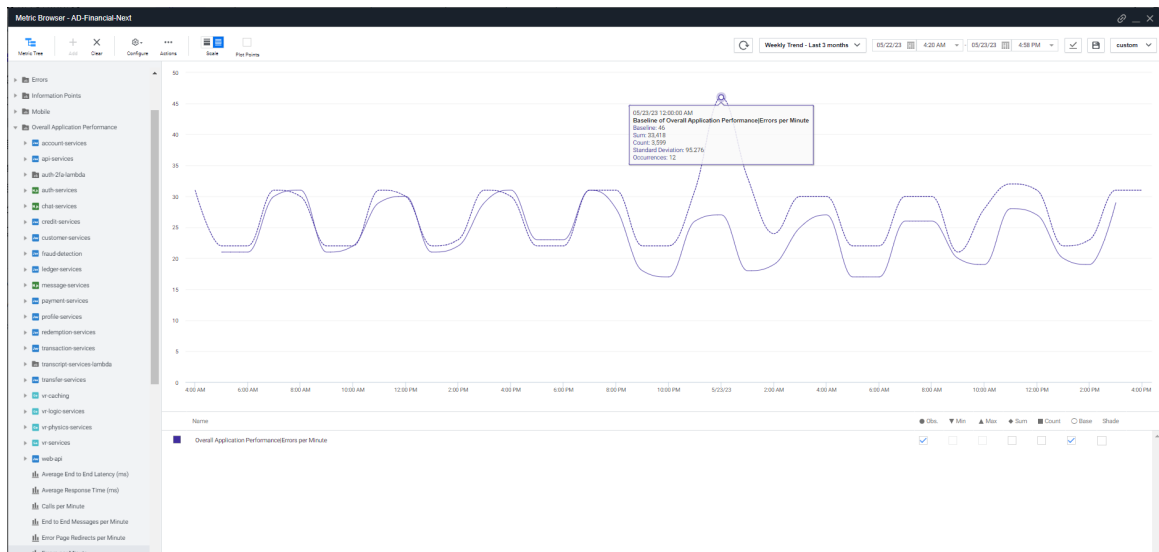


Figure 2.19 Dynamics Baseline view on metrics

Service Impact Solution

AppDynamics has an Automated Root Cause Analysis & Anomaly Detection feature, which will automatically notify the user if there are changes in anomalies and behavior in the application. AppDynamics will also provide root causes of problems that occur and also insight into service impacts related to ongoing problems.

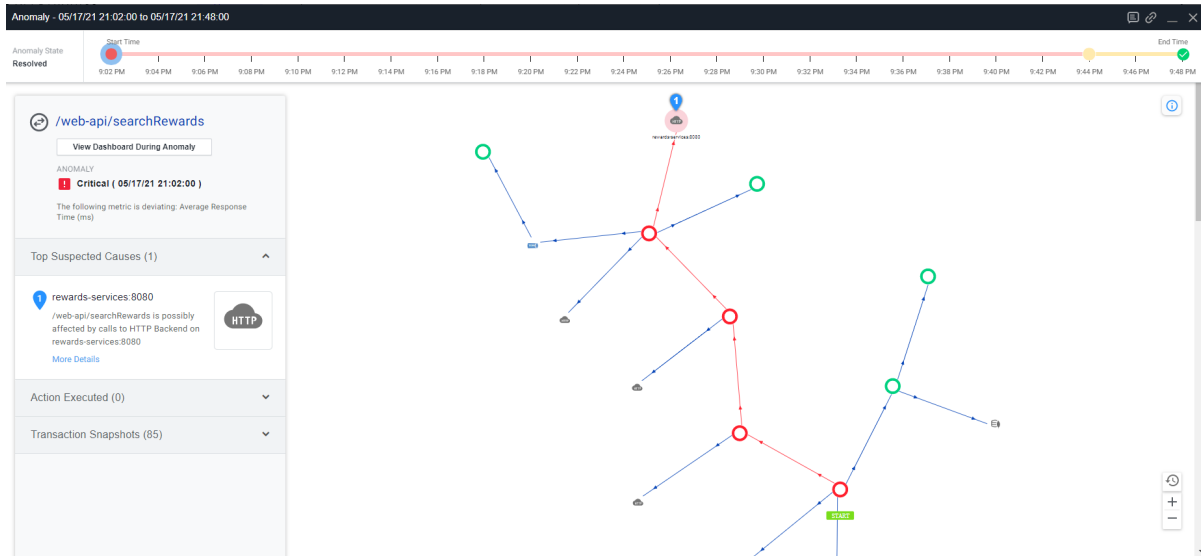


Figure 2.20 FlowMap Service Impact

Call Graph AppDynamics can also trace deep into the code-level which provides important information in the form of each service dependency required by the transaction, the percentage cost time consumed by each service and also if there are calls to third party calls or the database can show the correlation. to show the service that has the longest time consumption.

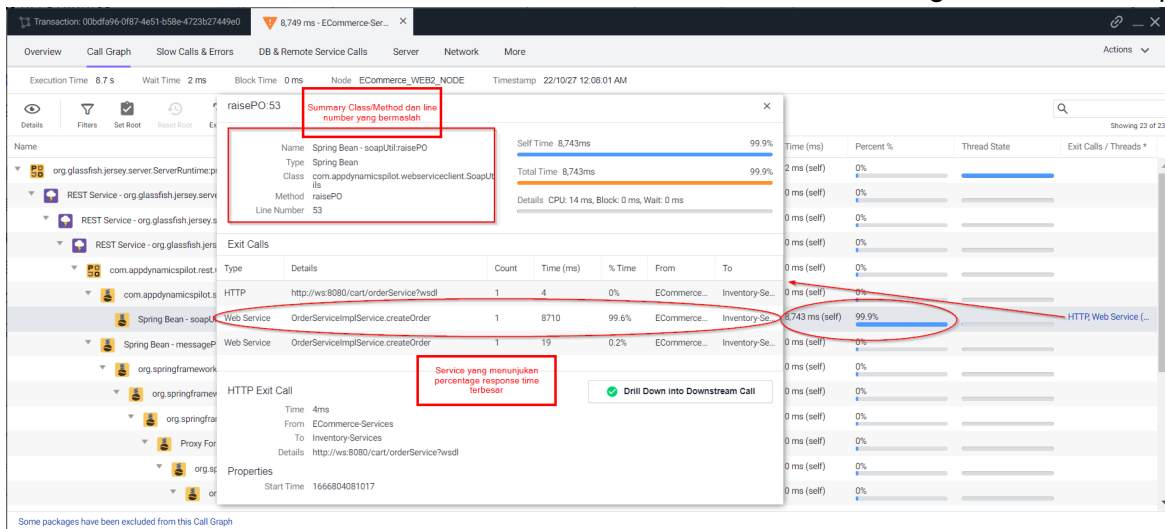


Figure 2.21 Detail Info Service Impact

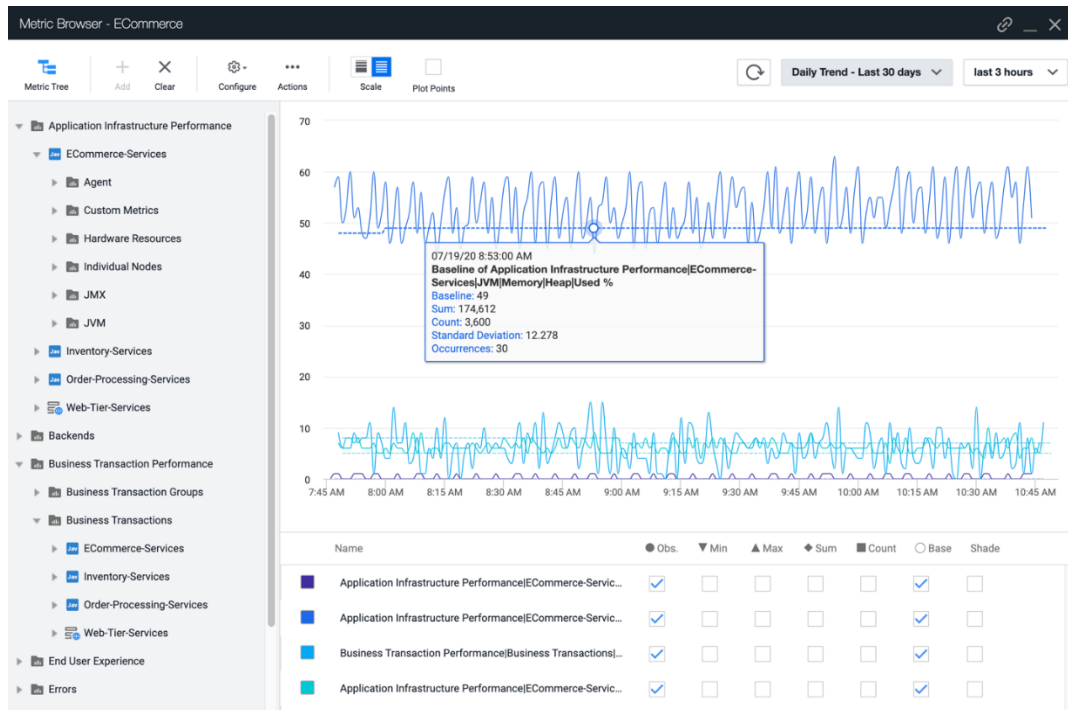


Figure 2.22 Metric Browser AppDynamics

Dashboard & Report

Dashboards and reports are very useful for getting a high-level understanding of the health and performance of the system regularly and in real time. The dashboard provides a graphical overview of selected data that can be accessed quickly. The dashboard itself can be custom-made, making it easy to create and organize widgets to provide users with visualizations of data that are interesting to see.

Custom Dashboards make it possible to display specific sets of metrics and data points into one view in real time. Custom Dashboards can display application, server, and database metrics collected by the AppDynamics Agent. We can also assign roles to specific users to see predefined custom dashboards. Every user who has the custom dashboard viewer role can view the custom dashboard.

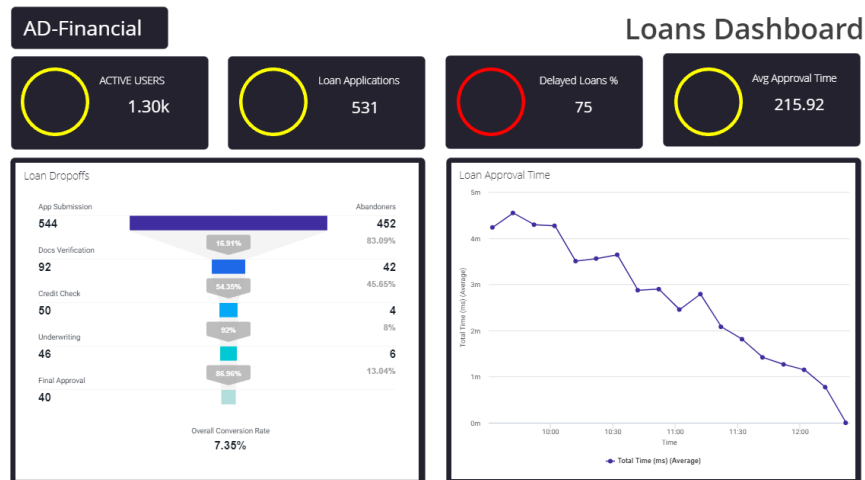


Figure 2.23 Custom Dashboard AppDynamics

Users can also customize charts according to data visualization needs. There are column, pie, table, tree map, area, line, points and distribution chart types.

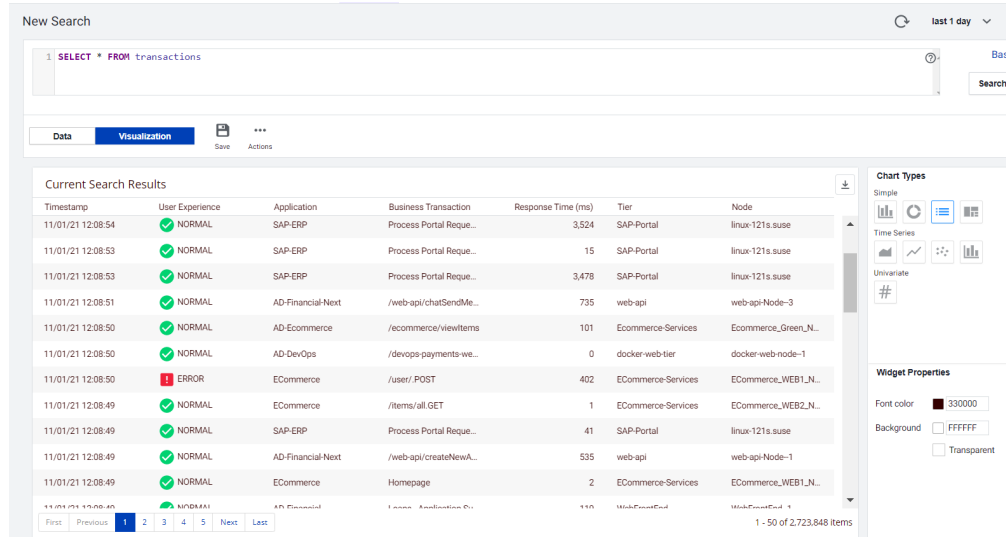


Figure 2.24 Custom Chart & Data

AppDynamics also has a Dash Studio feature, where users can easily create dashboards using existing widgets by drag and drop.

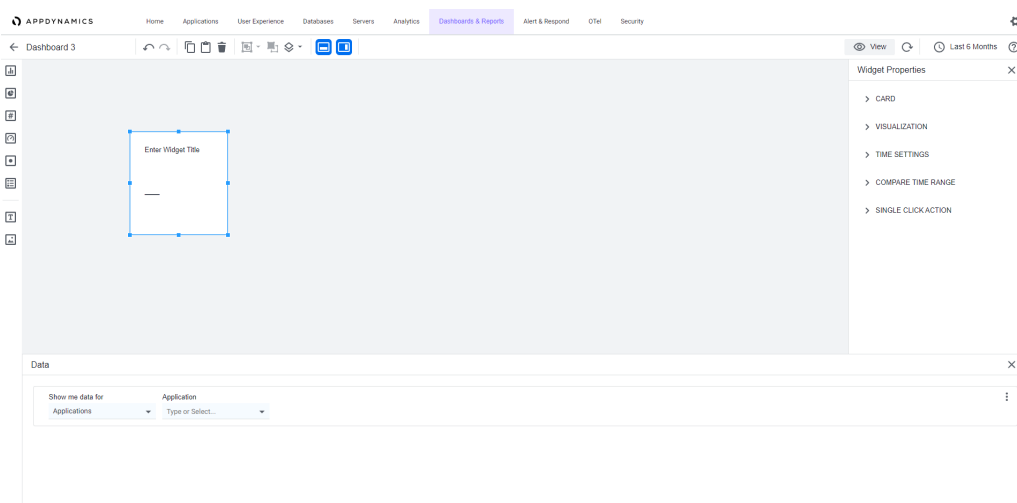


Figure 2.25 Menu Dash Studio

Report provides and shares AppDynamics data from the dashboard with recipients. Reports can be generated automatically on a regular schedule. Scheduled Reports are automatically generated at certain intervals. AppDynamics creates reports using data captured on the dashboard and sends them to each preconfigured email recipient. The following are the types of reports in AppDynamics.

Table 2.2 Table Reporting Matrix AppDynamics

Report Type	Captured Information
Application Health Report	Application health data from the application dashboard that includes: - Business transaction and server health data. - Transaction scorecard. - Events and exceptions. - Load, response time, and errors graphs.
Dashboard Report	Dashboard data in the selected custom dashboard.
Controller Audit Report	Audit entries that include: User logins and information changes.

	Controller configuration changes.	
	Environment properties changes.	
	Application properties and object changes such as policies, health rules, and the entities listed here:	
	Date and time range	ObjectType
	UserName	ObjectName
	Action	ApiKeyId (if applicable)
	ApplicationName	ApiKeyName (if applicable)
	AppDynamics supports PDF, JSON, and CSV file formats for this report type.	
Home Screen Report	Overview data from the home page.	
All Applications Summary	Data summary from the applications page.	
User Experience: Browser Apps	Browser RUM data from the Browser App Dashboard Overview page.	

AppDynamics reports can be generated using a time schedule. The generated report will be sent to email and in the form of a PDF file. The contents of the report can be generated using a custom dashboard. Figure 4.59 is an example of a report display sent via email in PDF file form.

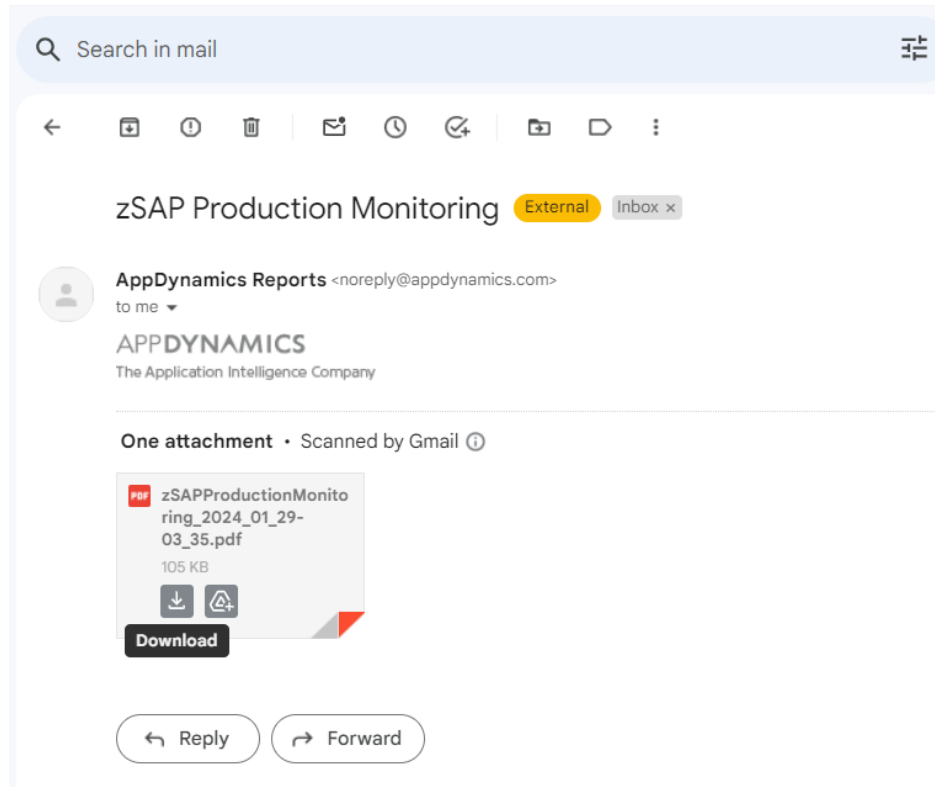


Figure 2.26 Report Notification

Notification

AppDynamics can deliver notifications or request actions based on configured conditions or events. By using the Alert & Respond feature, you can pinpoint problems as they occur, or even before they occur.

Policies serve as the central configuration artifact for alerting and response features. Policies allow you to define corrective actions to take when one or more conditions are met or an event occurs.

AppDynamics provides several pre-configured health rules that provide examples for creating other health rules. There are examples of health rules that can be customized to suit the performance requirements of the application being monitored.

Thresholds in Appdynamics can be based on static values or dynamic values. Thresholds in AppDynamics can be configured as needed. Threshold applies to the entire stack of applications or business transactions, including transactions that are in the background.

Thresholds can also be configured for diagnostic sessions. Thresholds can also be configured for slow, very slow or stalled transactions. When a transaction exceeds the threshold, AppDynamics starts taking snapshots. The snapshot contains a call graph for a transaction. The following is a picture of the alert & response topology in AppDynamics.

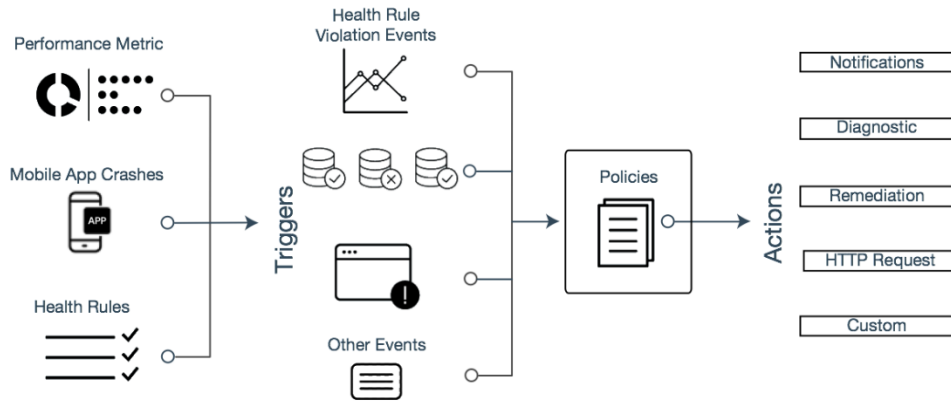


Figure 2.27 Flow Alert & Respond AppDynamics

There are several actions in AppDynamics that can be used as actions triggered by events or policies. Some actions that can be taken include sending emails, sending SMS messages, opening a diagnostic session, doing a thread dump, carrying out remediation such as executing scripts, creating or updating tickets in Jira or making HTTP requests. HTTP requests and webhooks can be used to integrate AppDynamics notifications with third applications such as WhatsApp, Tele-gram, Microsoft Teams, and others.

Figure 2.28 Action configuration view

Policies in AppDynamics can set triggering alerts and actions such as sending emails. Email recipients can be customized, adding and subtracting them according to your needs. Figure XX shows the display of adding emails for policies.

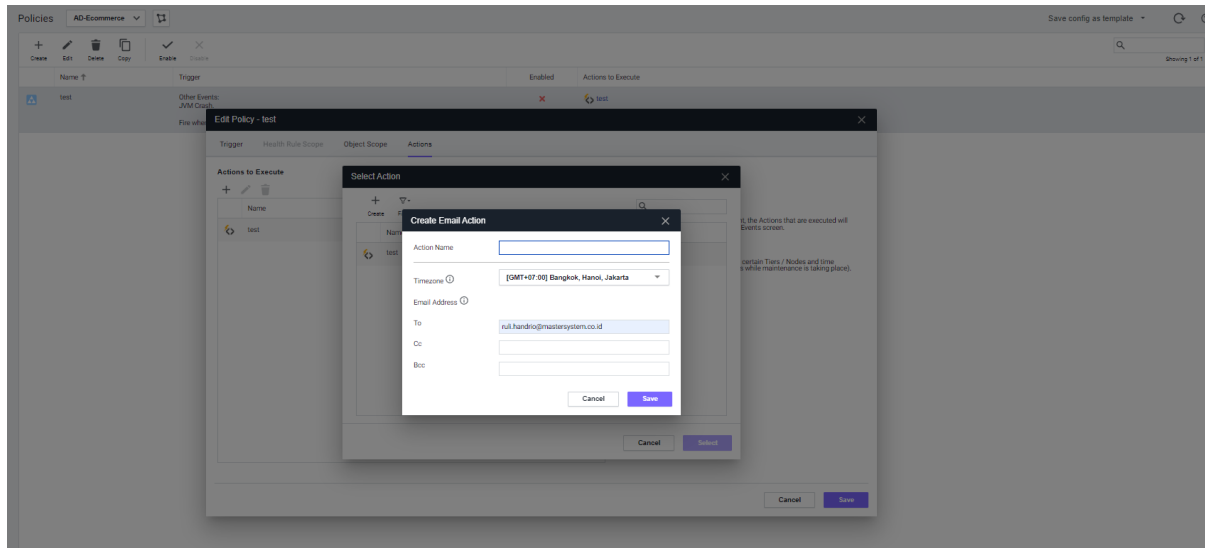


Figure 2.29 Notification Email configuration view

Retention Period

By default, AppDynamics Retention is able to store data for up to 365 days and is customizable. Users can back up data and expand it for longer storage.

Table 2.3 Retention Period AppDynamics

Property Name	About the property	Default
<code>appdynamics.controller.sim.purge.machine.using.metric</code>	Enable purging using only metadata information. Purging in this method does not use metrics data.	FALSE
<code>metrics.min.retention.period</code>	The number of hours 1-minute data is retained. This value should always be less than the value for <code>metrics.ten.min.retention.period</code> . Note that this property will vary in a non-timescale environment.	4 hours
<code>metrics.ten.min.retention.period</code>	The number of hours 10-minute data is retained. This value should always be greater than the value for <code>metrics.min.retention.period</code> and less than <code>metrics.retention.period</code> .	48 hours
<code>metrics.retention.period</code>	The number of days 1-hour data is retained. This value should always be greater than the value for <code>metrics.ten.min.retention.period</code> .	365 days
<code>purger.bts.shortlived.expected.baseline</code>	The expected number of short-lived business transactions to be purged in one purging attempt.	100

<code>purger.metrics.shortlived.expected.baseline</code>	The number of entities the short-lived metric purger expects to be purged on every purging attempt.	1000
<code>shortlived.metric.purger.time.interval.in.min</code>	The interval frequency in minutes for the short-lived stale metric purger timer task to run.	30
<code>shortlived.metric.purging.enabled</code>	Enable the short-lived metric purger. When set to true, the short-lived metric purger timer task purges all stale metrics which are short-lived. For example, container metrics.	TRUE
<code>shortlived.stale.metric.duration.in.hours</code>	The duration of data in an hour, in which a metric that has not reported any data, is considered stale.	2

Historical data in AppDynamics can be copied or pulled using the REST API in JSON form. Figure 4.66 shows specific metrics with customized time range historical data of 8 months.

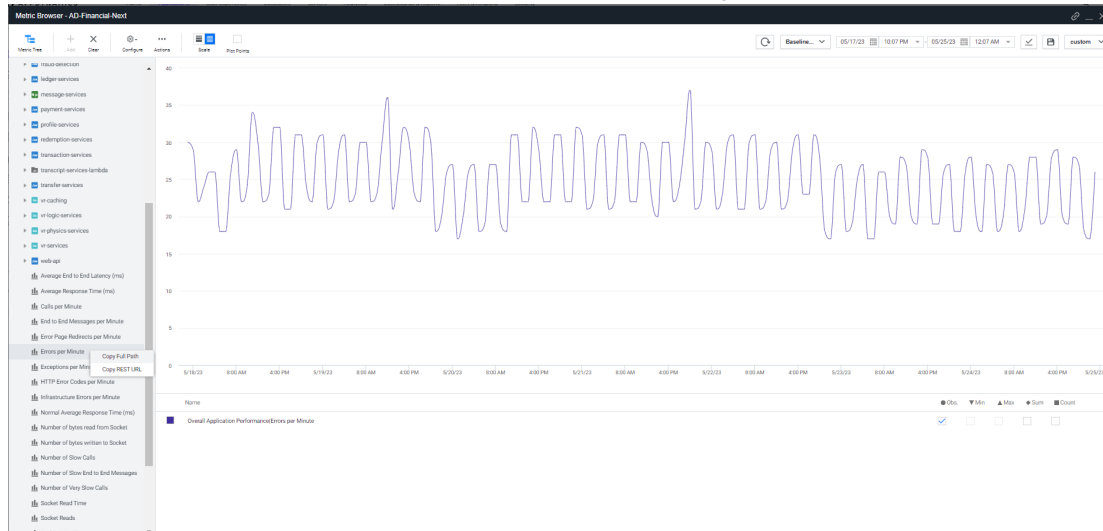


Figure 2.30 Metrics Browser displays historical data

AppDynamics APIs

AppDynamics has prepared APIs that can be used to add capabilities to ap-pdynamics. There are several types of APIs that can be used, such as Controller APIs, Application APIs, Configuration APIs, Database Visibility APIs, Analytics Event APIs, Metrics and Snapshot APIs, RBAC APIs, Controller Audit History APIs, Synthetic Monitoring APIs, and others. AppDynamics provides diagnostic features that are useful for capturing in more detail and obtaining more information. Diagnostic is shown when the user wants to carry out more detailed tracing which requires a lot of data samples with complete information.

Integration

AppDynamics has prepared several integrations to third-party applications to increase the capabilities of AppDynamics itself. Some integrations are as follows.

1. CompuWare Strobe
2. DB CAM
3. Jira Action
4. Log Auto-Discovery Extension
5. Rookout
6. Scalyr
7. ServiceNow
8. Splunk

AppDynamics also have more than 100 of extension to provide integration or monitoring for third-party application such as F5, Redis, zookeeper, Kafka and etc.

Security

For security purpose, AppDynamics Agent is using SSL TLS 1.3 for connectivity from agent to the controller. For Isolated application, AppDynamics agent can connect to the SaaS controller using proxy. Security Pages AppDynamics requires user credentials to be able to access AppDynamics which requires authentication using a password.

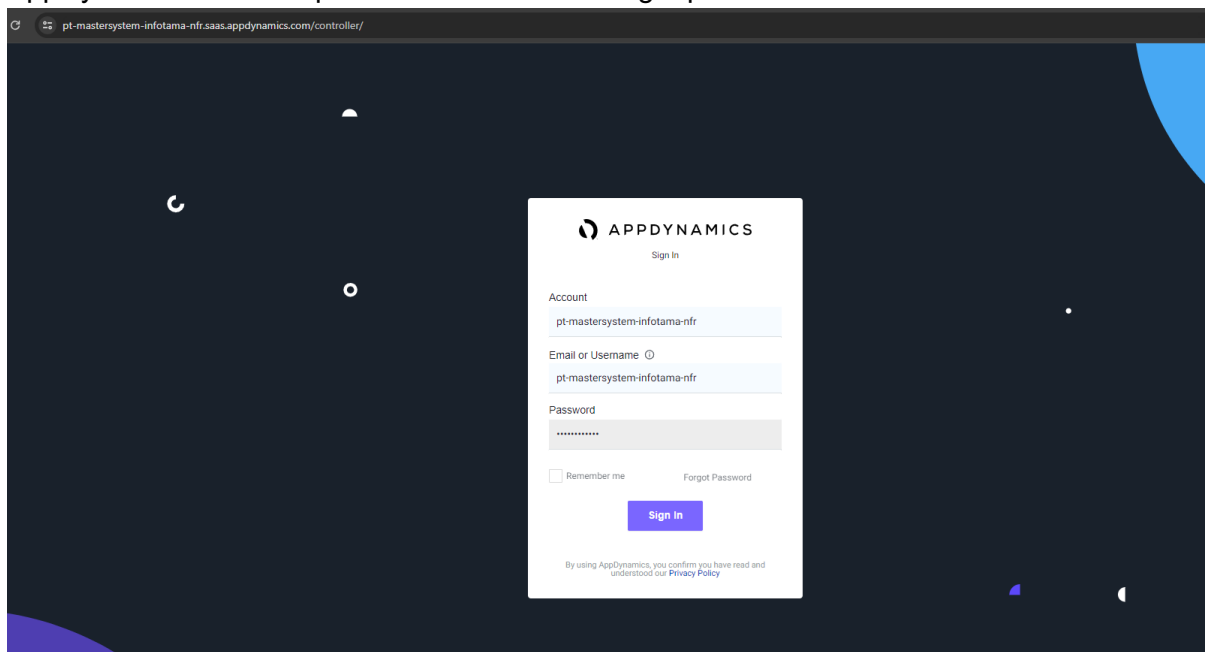


Figure 2.31 Appdynamics login homepage display

By default, AppDynamics does not use strong passwords, if needed there is a strong password feature in the administration menu.

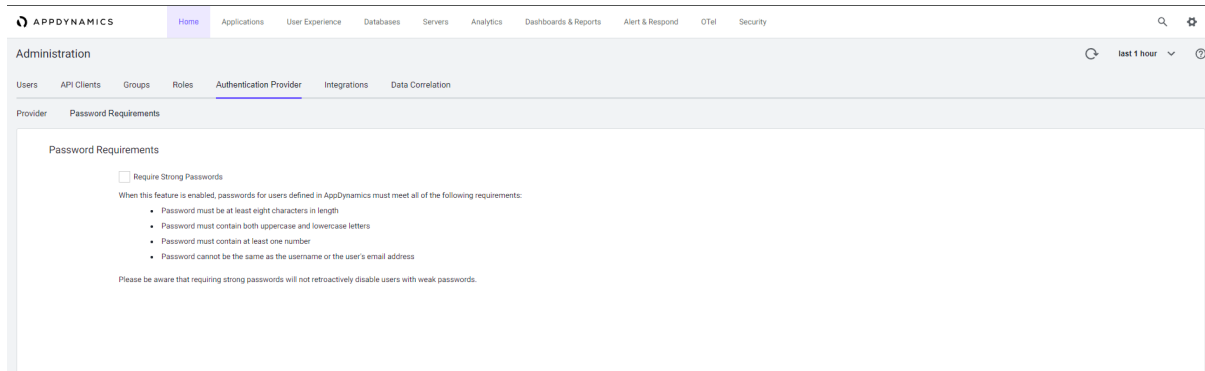


Figure 2.32 Configuration to force the use of strong passwords

Supported Technologies

The following is a list of supported technologies for AppDynamics.

Table 2.4 List of Cisco AppDynamics Supported Technologies

Programming Language & Technology	Agent	Status
Java	Java Agent	✓
Scala	Java Agent	✓
.Net Framework/.Net Core	.Net Agent	✓
Node.js	Node.js Agent	✓
PHP	PHP Agent	✓

Table 2.5 Advanced List of Cisco AppDynamics Supported Technologies

Programming Language & Technology	Agent	Status
Python	Python Agent	✓
AWS Lambda	Serverless APM	✓
Apache Web Server	Apache Web Server Agent	✓
C/C++	C/C++ SDK	✓
Go Lang	Go Language Agent	✓
Ruby	Ruby Agent	✓
SAP	ABAP Agent	✓
IBM Integration Bus	IBM Integration Bus Agent	✓
Swift	Mobile Agent	✓
Shell Scripting	Machine Agent	✓
HTML/CSS	Javascript Agent	✓

There are still many other technologies that AppDynamics supports. You can see the whole thing in the AppDynamics documentation.

AppDynamics can also use custom metrics. AppDynamics provide Monitoring extension script (also known as a custom monitor or hardware monitor) to add custom metrics to the metric set that

AppDynamics already collects and reports to the Controller. Your script reports the custom metrics every minute to the Machine Agent. The Machine Agent passes these metrics to the Controller. If the application using iso8583 for their format when send/receive request than we can use data collector to capture ISO8583 messages

AppDynamics also can leverage their capabilities to monitor synthetics monitoring with ThousandEyes to monitor HTTP performance, DNS server performance, path visualization, FTP performance and so on.

AppDynamics also provide extension to monitor third-party application and several technologies through extension. More than 90++ extension provided like MQ extension, ELK Extension, F5 extension and etc.

RoadMap AppDynamics

AppDynamics will continue to develop the platform to be able to comply with the latest technology used both in terms of flexibility and performance. AppDynamics is currently included in Cisco's FSO (Full Stack Observability) Platform. Where this platform can provide a complete observability solution for users, starting from end-user performance, internet performance, CDN, DNS, path visualization, frontend, backend, application performance, business performance, code visibility, infrastructure, network performance, security to automation. optimization. The following is a big picture regarding Full Stack Observability from Cisco.

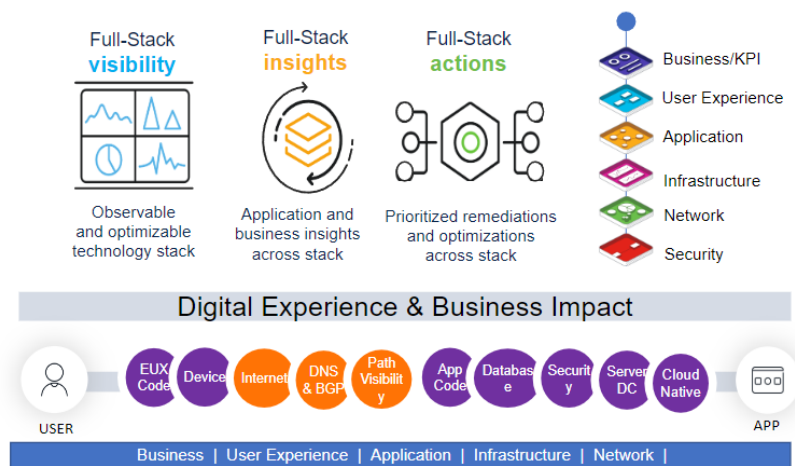


Figure 2.33 Full Stack Observability Cisco

Business Value Analysis

AppDynamics offers prospective customers the opportunity to carry out a Business Value Assessment (BVA). The result of this approach is a business case that is aligned with the Proof of Value (PoV) and describes the advantages you want to expect when choosing AppDynamics as a solution.

The business case is built from data collected from various one-on-one meetings with relevant stakeholders. The meeting usually discusses the context of the AppDynamics solution to qualify ('AS Is') the challenges faced by each different team, identify how AppDynamics can answer this ('To Be') and use the data from the PoV results with previous experience, then also quantify

the potential benefits of using AppDynamics. Benefits are usually described in terms of increased revenue, cost reduction, cost avoidance or risk reduction for customers. Post-sale, Business Value practice uses value cases which create business cases with our Customer Success Manager to periodically measure expected profits that have been collected and identified

Report in May 2018, "The Total Economic Impact™ Of AppDynamics Application Performance Monitoring with Cisco" Forrester stated that the AppDynamics APM solution reduced issues by approximately 95% in the production environment. Other key metrics presented in the report were:

- IT operations time and help desk time savings: 65% reduction in repairs made by IT operations, and 30% reduction in calls to the IT help desk
- Application development cycle time savings: 45% reduction in application development time and effort.
- Application monitoring and detection tool consolidation: 70% reduction in monitoring tools

Table 2.6 Table Case Studies

Area of Improvement	Quoted Improvement	Reference
Incident Reduction	30% Reduction in Incidents	· Barclays - 30% Reduction - https://www.computerworld.com/article/3427420/application-incidents-down-30-percent-at-barclays-following-appdynamics-adoption.html · Alaska Airlines - 60% Reduction - https://www.appdynamics.com/case-study/alaska-airlines · Forrester Report - Page 13 - Reduction in IT Calls by 30%
MTTR Reduction	75% Reduction	· Overstock - Days to Minutes - https://www.appdynamics.com/case-study/overstock · T Systems - Minutes - https://www.appdynamics.com/case-study/t-systems

		· Forrester Report - Page 12 - MTTR has been reduced in the range of 50% to 70%.
Root Cause Analysis Effort	75% Reduction	· Covis - 5 to 10 Minute Root Cause Time - https://www.appdynamics.com/case-study/covis
Developer Effort per Performance Testing Defect	50% Reduction	· Forrester Report - Page 15 - Application development time decreased by 45% with AppDynamics. · Telenor - Development - https://www.appdynamics.com/case-study/telenor
Release Performance Validation	50% Reduction	· Forrester Report - Page 16 - 45% Acceleration in app or functionality rollout, counted in percentage decrease in development time · Telenor - Development - https://www.appdynamics.com/case-study/telenor
Poor Performing Application Minutes	70% Reduction	· Infomedia - 80% Application Performance Improvement - https://www.appdynamics.com/case-study/infomedia